Introduction to High-Performance Machine Learning

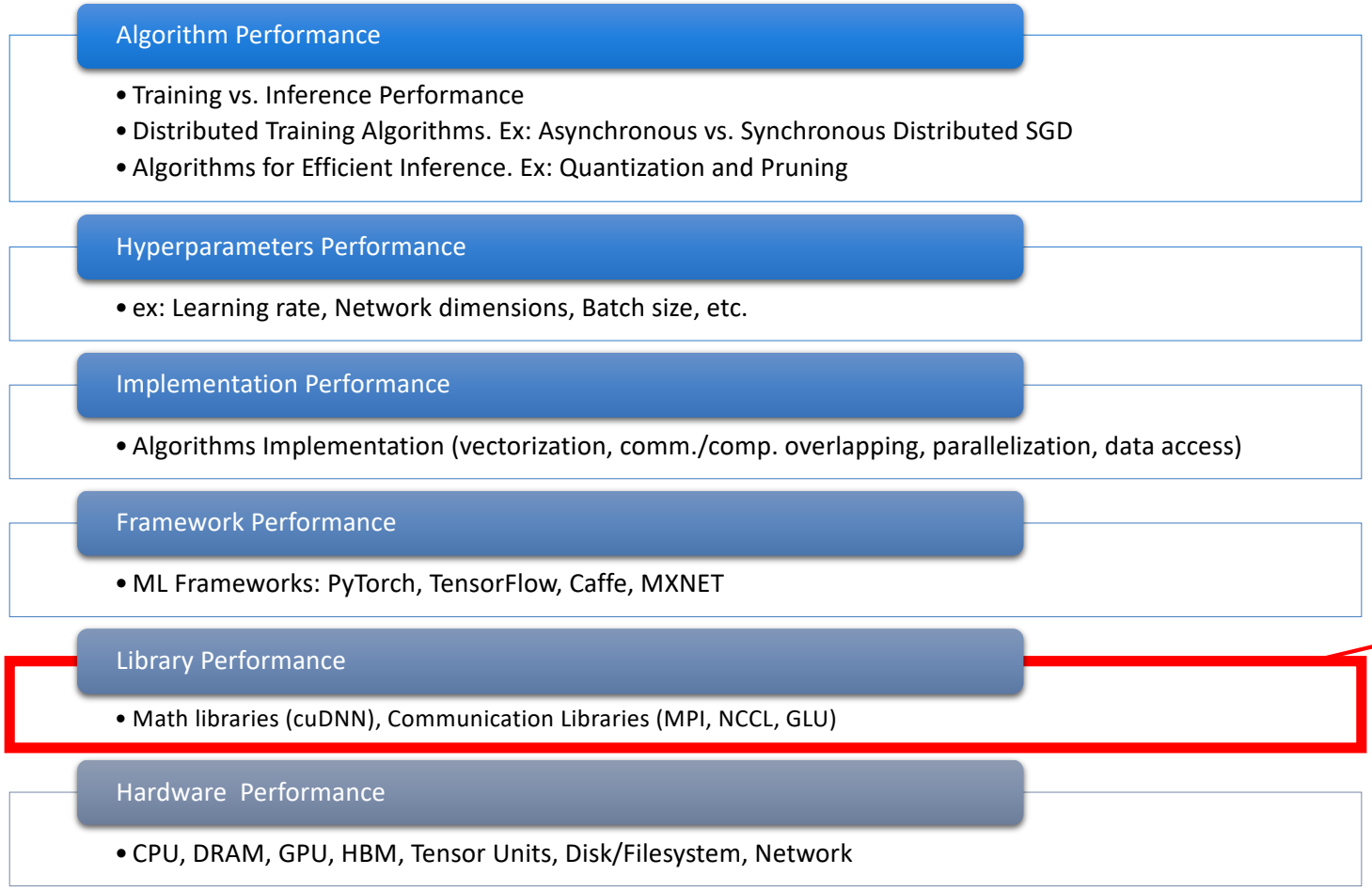
Lecture 4 02/23/26

Dr. Kaoutar El Maghraoui

Attendance QR Code

CUDA Basics

ML Performance Factors



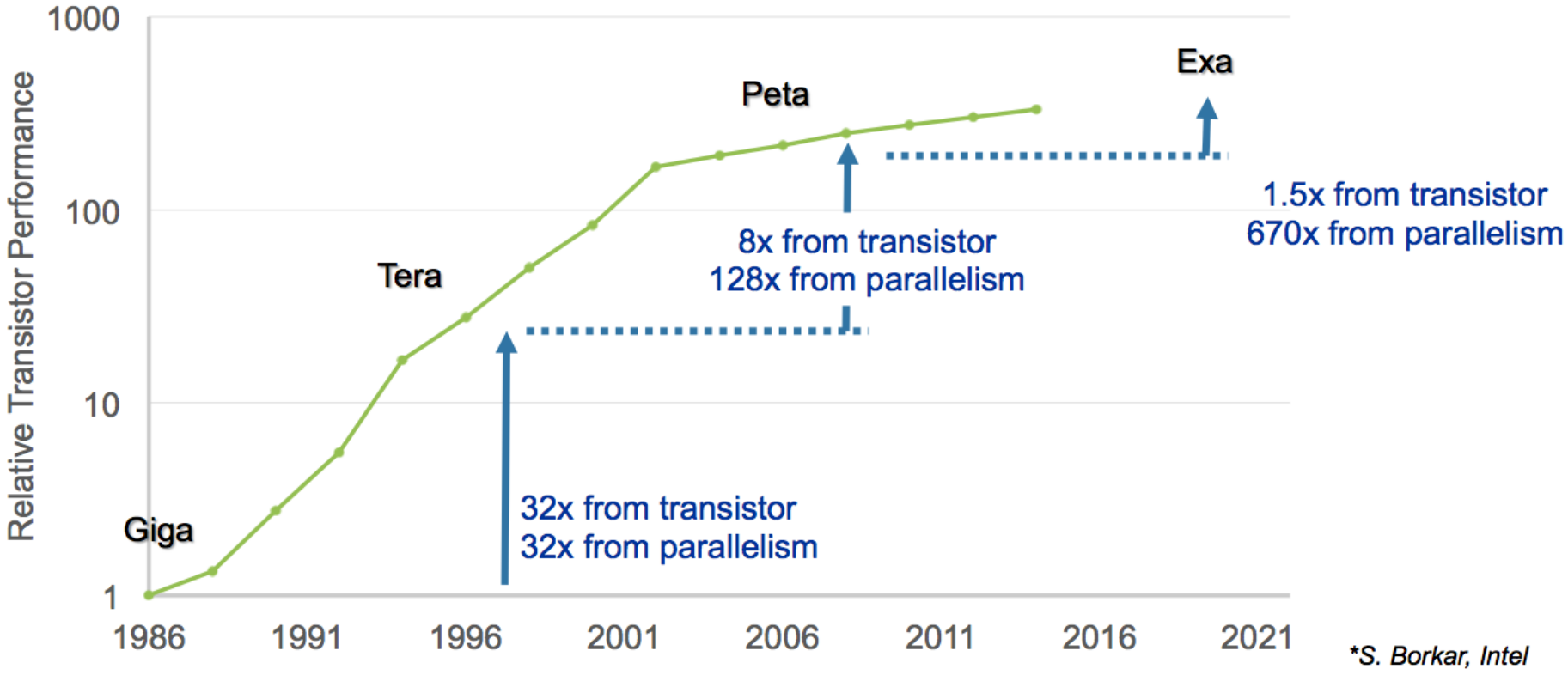
Focus for today's lecture

Lesson Outline

- Heterogenous architectures motivations
- Hierarchy of Computations:
  - Threads
  - Blocks
  - Grids
- Corresponding Memory Spaces
  - Local
  - Shared
  - Global
- Synchronization Primitives
  - Implicit Barriers
  - Thread Synchronization
- NVIDIA GPUs and CUDA:
  - Compute capability
- CUDA Programming Model:
  - Grid, Block, Thread
  - UVM
- CUDA Hardware
- CUDA Warp Scheduling
- Context and Stream
- CUDA Memory Alignment and Coalescing
- CUDA Profiling and Debugging
- CUDA Compilation and Runtime:
  - CUDA Runtime, CUDA Driver, AoT and JIT compilation

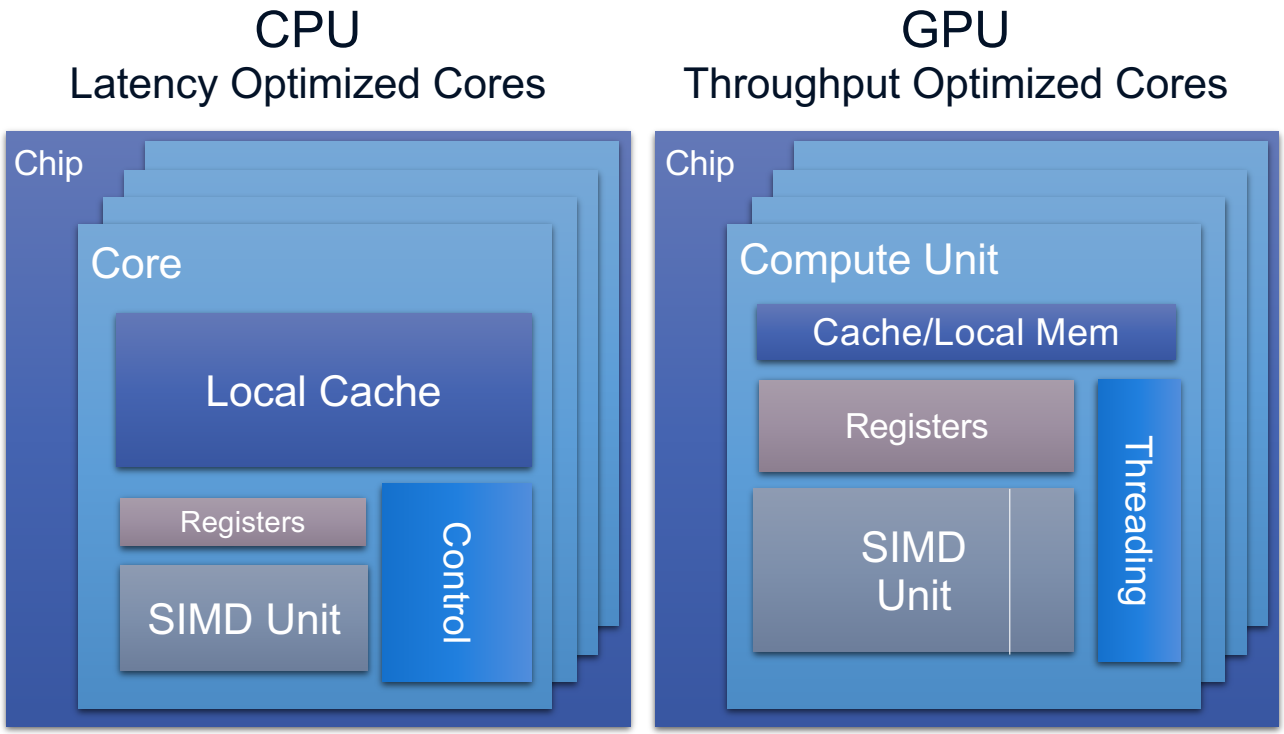
Heterogenous computing motivations

Heterogeneous Computing Motivation



Difference between CPU and GPU

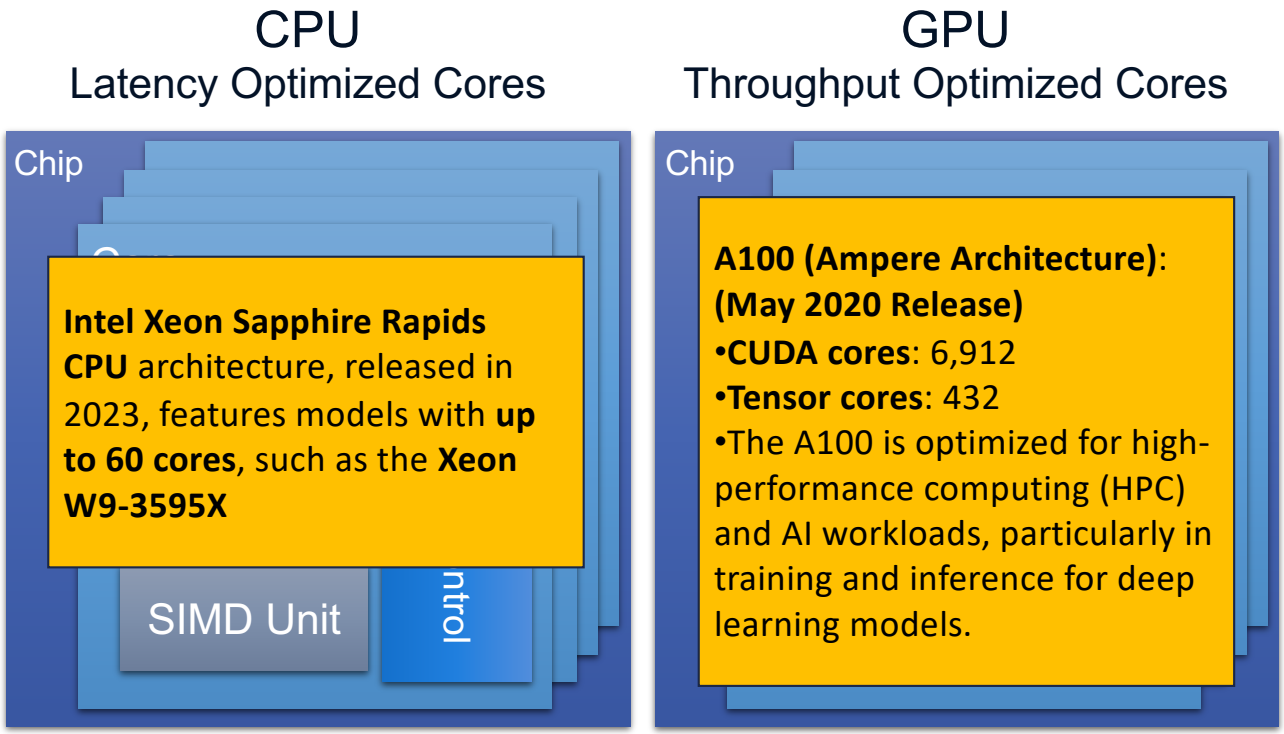
- Latency Optimized:
  - Optimized to compute operation in a minimum time
- Throughput Optimized:
  - Optimized to compute many operations on the same time



From: Nvidia – U Illinois

Difference between CPU and GPU

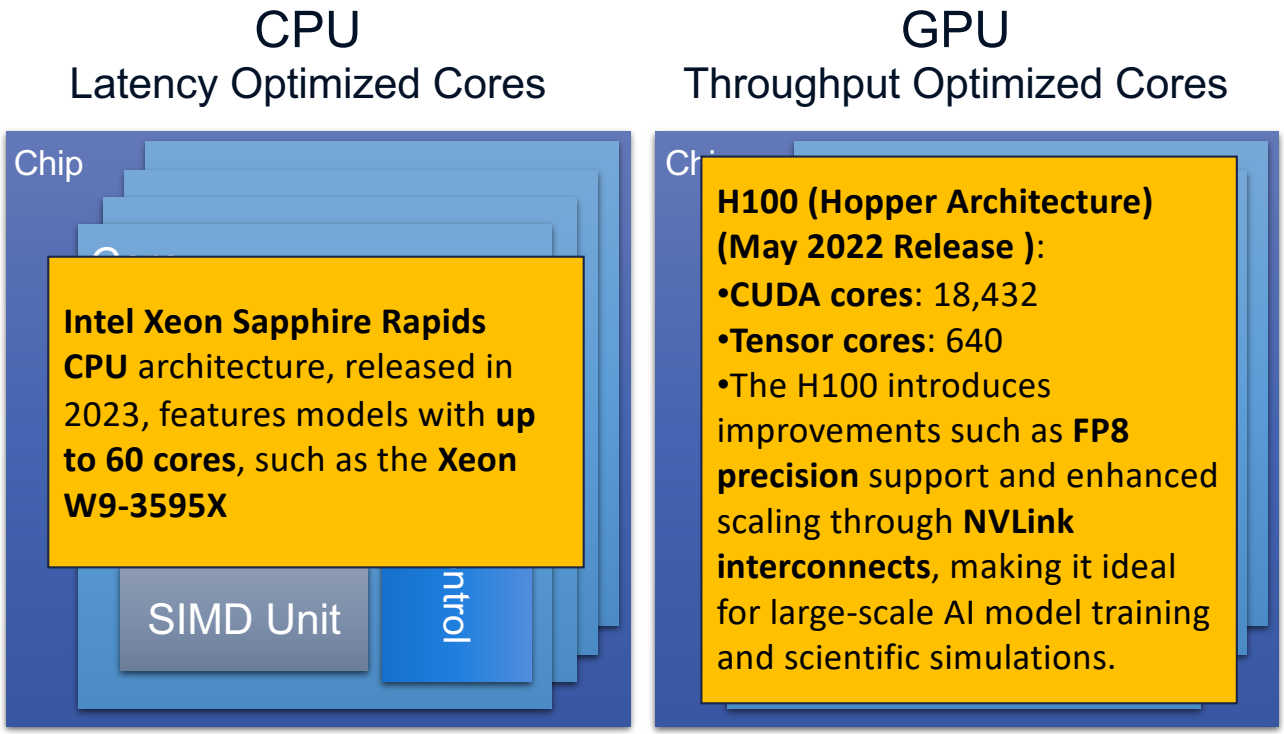
- Latency Optimized:
  - Optimized to compute operation in a minimum time
- Throughput Optimized:
  - Optimized to compute many operations on the same time



From: Nvidia – U Illinois

Difference between CPU and GPU

- Latency Optimized:
  - Optimized to compute operation in a minimum time
- Throughput Optimized:
  - Optimized to compute many operations on the same time



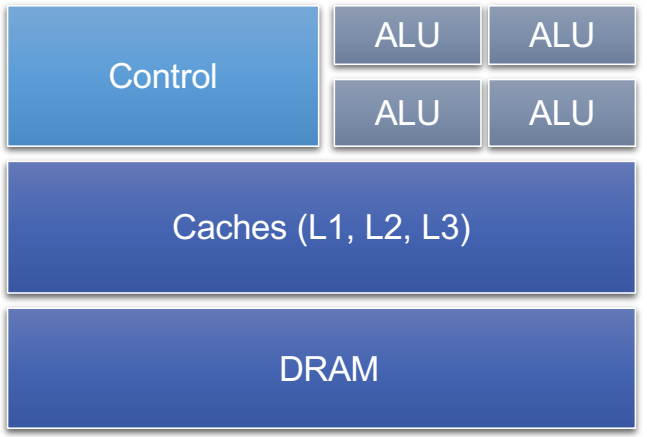
From: Nvidia – U Illinois

CPU: Latency Optimized

- Powerful **Arithmetic Logic Unit**
  - Optimized for latency
- Sophisticated **Control logic**
  - Branch prediction for reduced branch latency
  - Data forwarding for reduced data latency
- Large **L1, L2, L3 Cache** Hierarchy
  - Convert long latency memory accesses to short latency cache accesses

- https://cacm.acm.org/magazines/2010/11/100622-understanding-throughput-oriented-architectures/fulltext

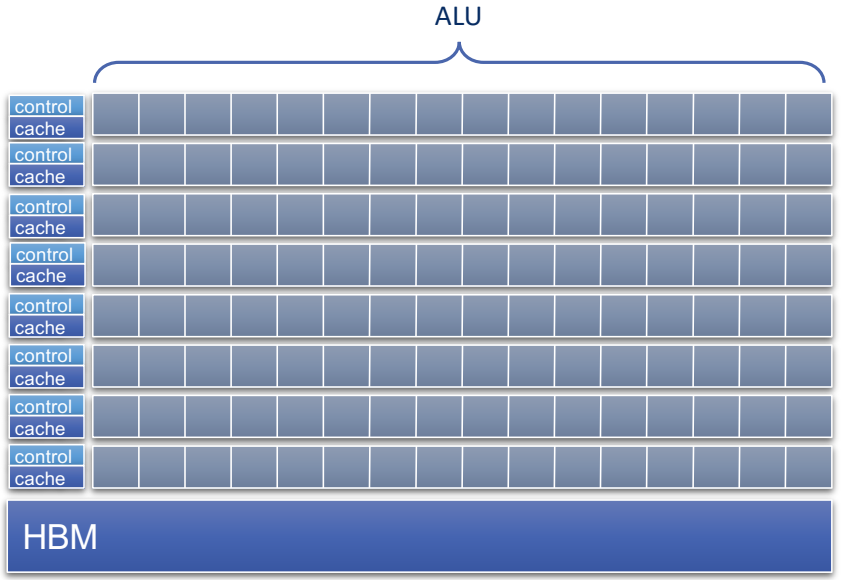
CPU Architecture



GPU: Throughput Optimized

- Many small **ALU**s
  - Many
  - Long latency
  - Parallelism
- Small **Caches (shared memory)**
  - To boost memory throughput
- Simple **Control**
  - No branch prediction
  - No data forwarding
  - Require massive number of threads to tolerate latencies
  - Threading logic and state

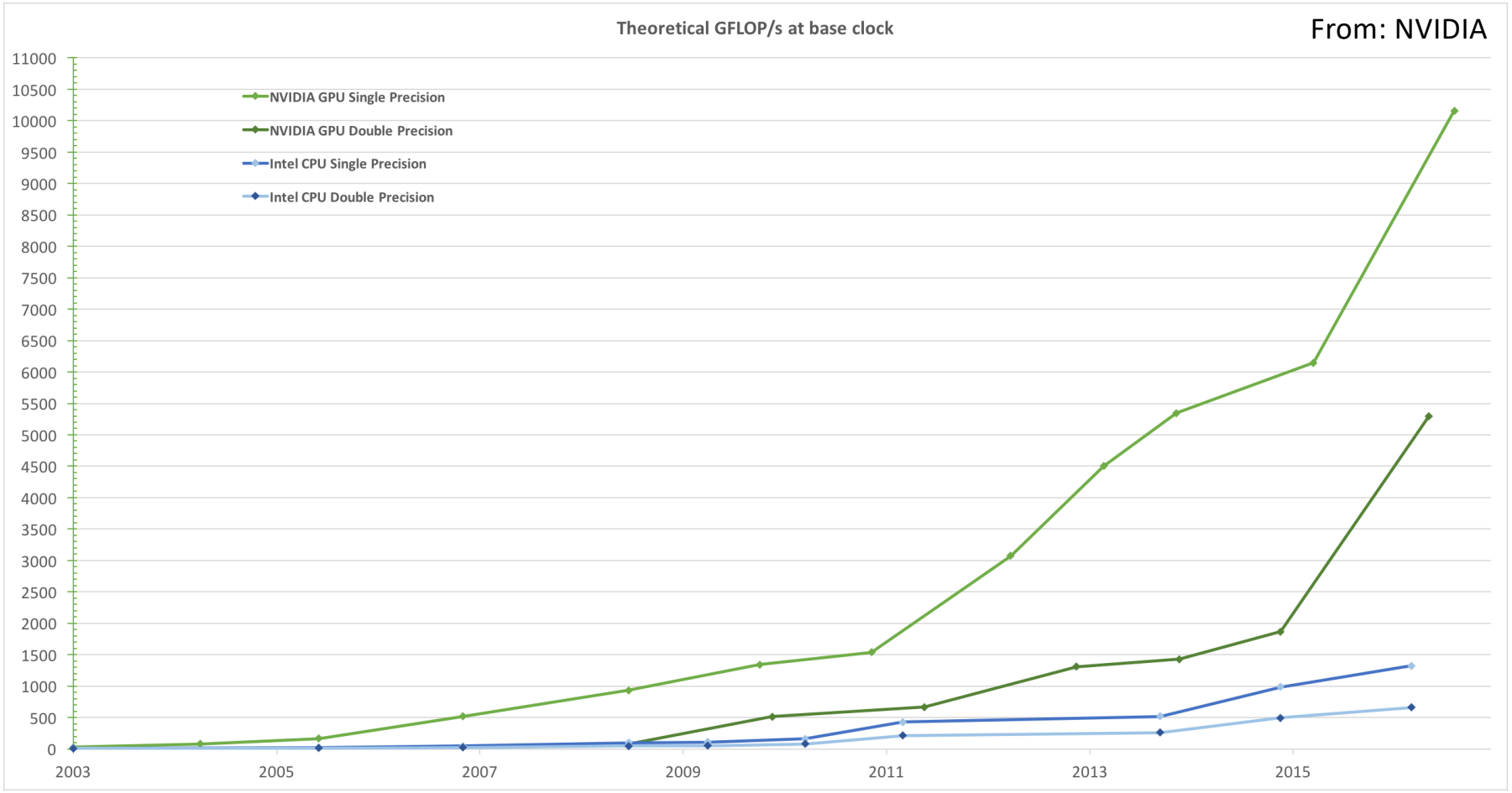
GPU Architecture



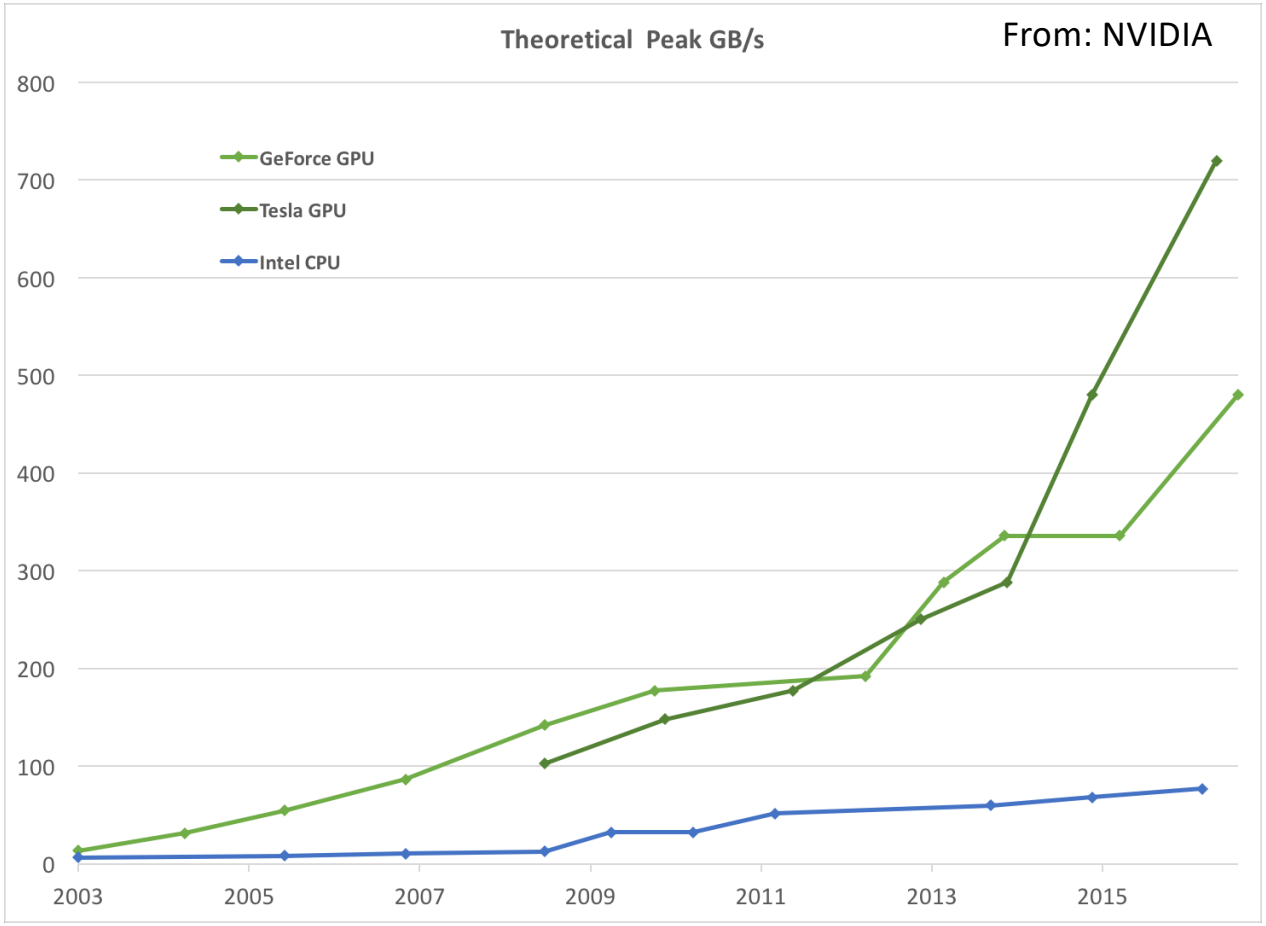
CPU/GPU Performance Comparison

- Sequential Code:
  - CPU about 10x faster than GPU
- Parallel Code:
  - GPUs can be 10X+ faster than CPUs for parallel code
- Latencies:
  - CPU: latency of operations is in **nsec**
  - GPU: launching a kernel can take 10s **micro-sec** or more

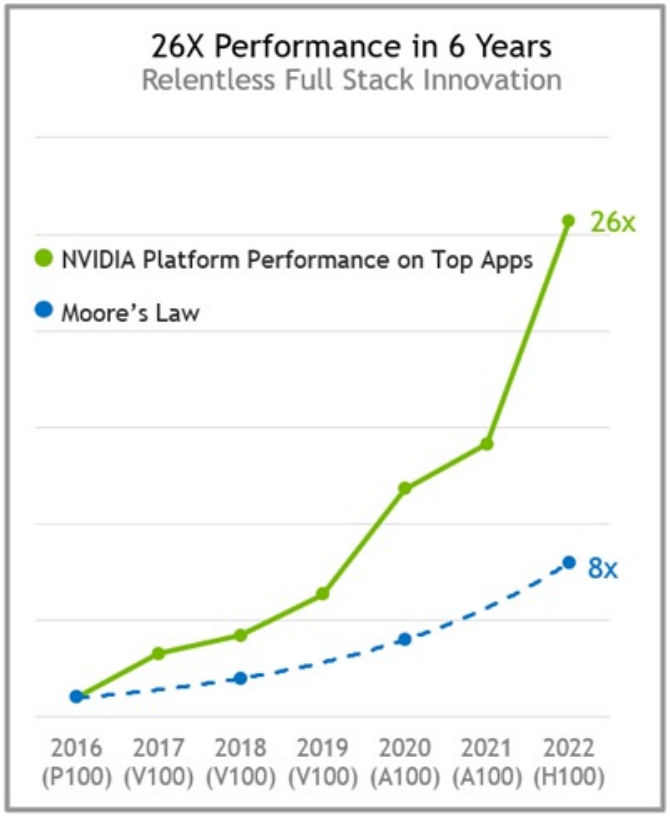
GPU Performance Advantage 1: FLOPS

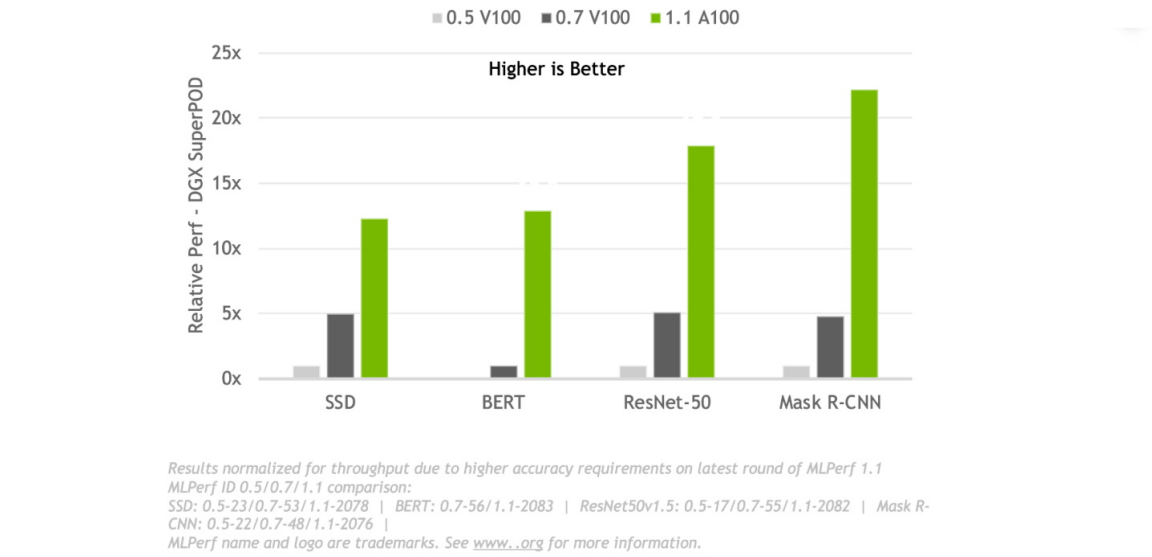


GPU Performance Advantage 2: Memory BW



NVIDIA GPU Performance Improvements Over the years





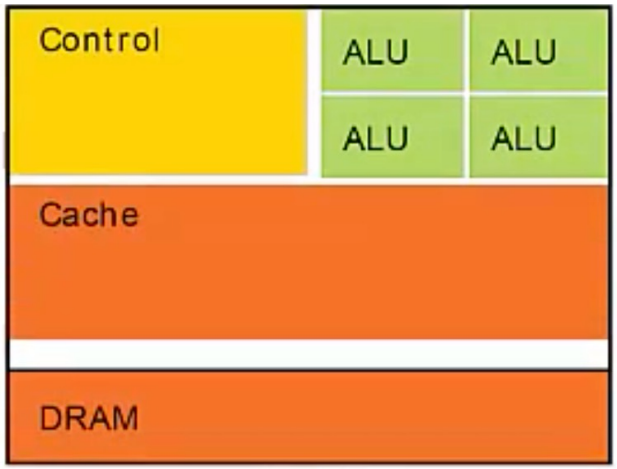
Source: https://developer.nvidia.com/blog/fueling-high-performance-computing-with-full-stack-innovation/

CPU vs. GPU Summary

CPU	**GPU**
A smaller number of larger cores (up to 24)	A larger number (thousands) of smaller cores
Low latency	High throughput
Optimized for serial processing	Optimized for parallel processing
Designed for running complex programs	Designed for simple and repetitive calculations
Performs fewer instructions per clock	Performs more instructions per clock
Automatic cache management	Allows for manual memory management
Cost-efficient for smaller workloads	Cost-efficient for bigger workloads

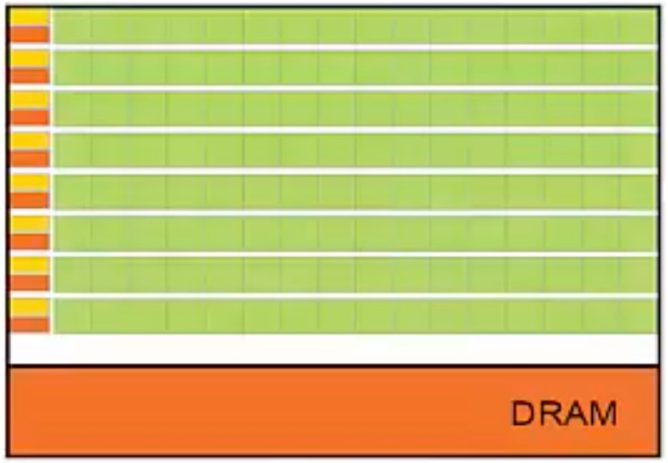
CUDA Programming Model

Difference between CPU's and GPU's



CPU

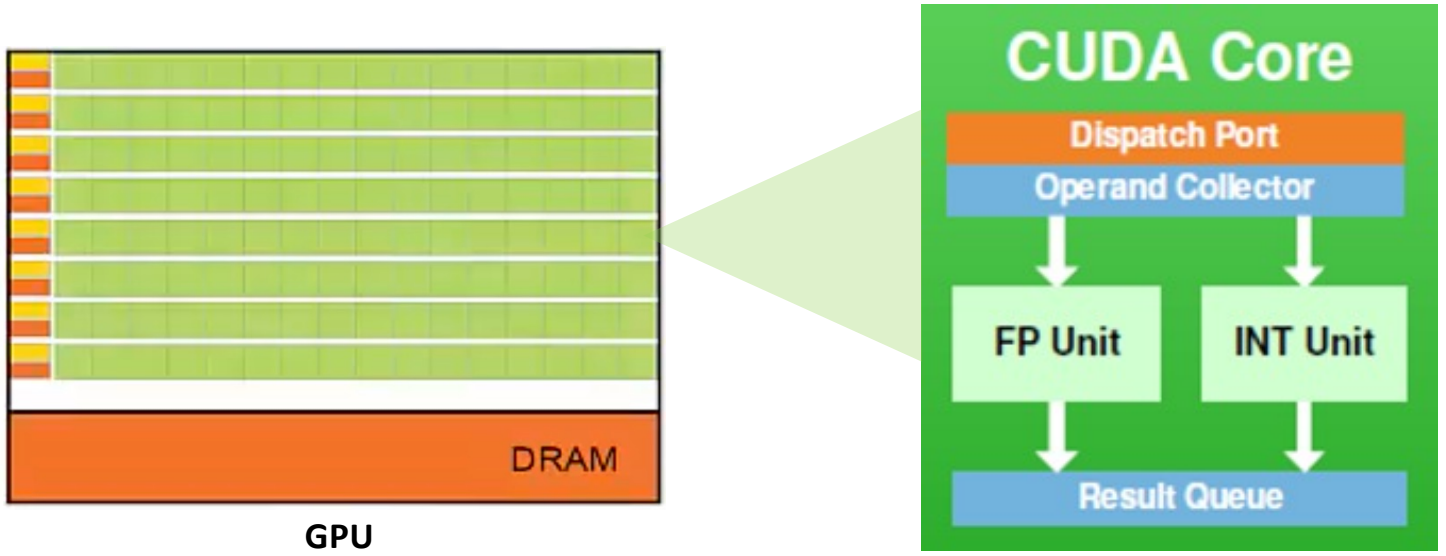
- CPUs have few strong cores
- CPU- Designed to minimize latency
  - Majority of the silicon area is dedicated to:
    - Advanced Control Logic
    - Large Cache



GPU

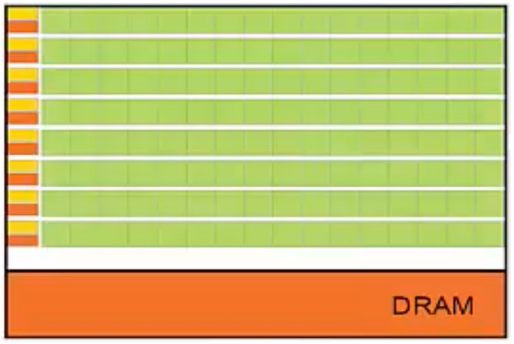
- GPUs have thousands of weaker cores
- GPU- Designed to maximize throughput
  - Majority of the silicon area is dedicated to:
    - Massive number of arithmetic units – CUDA cores

What is a CUDA Core



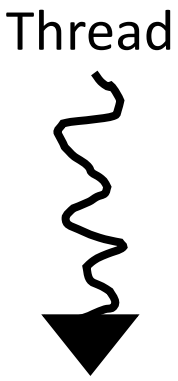
Key Concepts

To properly utilize the GPU, the parts of the program that can be parallelize must be decomposed into a large number of threads that can run concurrently in CUDA.

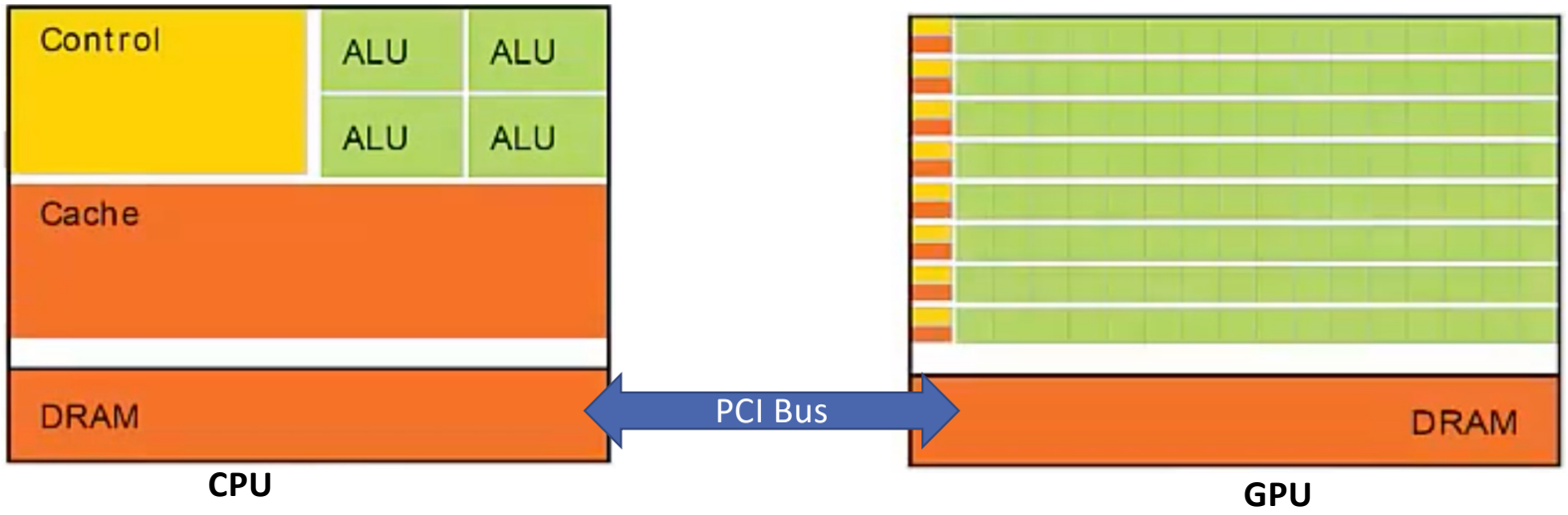


GPU

Kernels	Threads
• Functions that run on the GPU • Kernel are lunched on the CPU	• Kernels execute as a set of parallel threads • Each thread is mapped to a singled CUDA core on the GPU



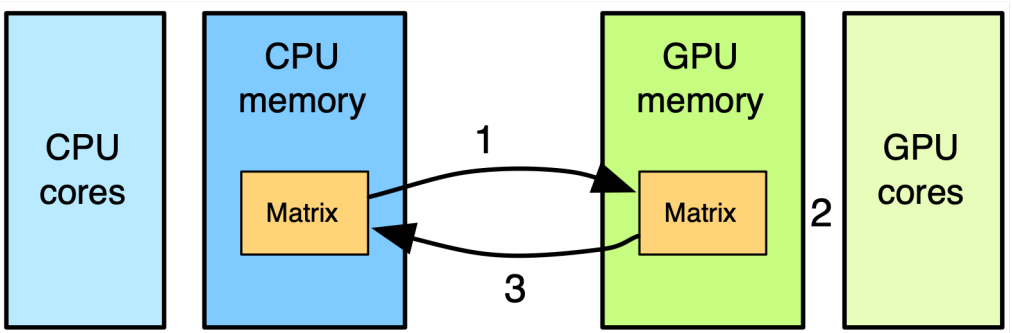
Key Concepts



Host	Device	Heterogenous	CUDA
• CPU and its on chip memory	• GPU + dedicated DRAM	• Host + Device • Take the best of both worlds	• C with extensions • Allows for heterogenous programming

GPU Computing

- Computation is offloaded to the GPU
- Three steps
  - CPU-GPU data transfer (1)
  - GPU kernel execution (2)
  - GPU-CPU data transfer (3)



Operations Performed on the CPU

- **Data Loading and Preprocessing**:
  - The CPU handles **loading data** from storage (e.g., datasets or batches) and performs **preprocessing** tasks like data augmentation, normalization, shuffling, and tokenization for NLP models.
  - Tools like **PyTorch's DataLoader** or **TensorFlow's tf.data** pipelines run on the CPU to prepare input data.
- **Model Initialization and Control Logic**:
  - CPUs are responsible for **initializing models** (e.g., setting up weights and biases) and managing the **control flow** of training (e.g., loop iterations and logging).
- **CPU-Side Operations and Mixed Precision Coordination**:
  - Mixed precision training (using both FP32 and FP16 precision) requires **precision switching coordination**, which is often managed by the CPU.
  - CPUs also **manage kernel dispatching** for GPUs.
- **Gradient Aggregation (in Multi-GPU Training)**:
  - In distributed settings, the CPU may aggregate gradients from multiple GPUs and perform **synchronization**using frameworks like **NCCL** or **MPI**.
- **Saving and Loading Models**:
  - CPUs are used to **serialize and deserialize models** (e.g., saving checkpoints to disk or loading pretrained models).

Operations Performed on the GPU (1/2)

- **Forward Pass (Inference)**:
  - The **GPU computes the forward pass** by executing matrix multiplications (e.g., **dense layers, convolutions**) in parallel, making it ideal for workloads like image recognition or language model inference.
- **Backward Pass (Gradient Computation)**:
  - The **GPU handles backpropagation**, computing the **gradients for each layer** during training. Matrix operations involved in calculating gradients benefit from the GPU's parallel nature.
- **Weight Updates (Training)**:
  - Once gradients are computed, **weight updates** (e.g., using **SGD** or **Adam** optimizers) can also be done on the GPU for better performance, particularly in large models.

Operations Performed on the GPU (2/2)

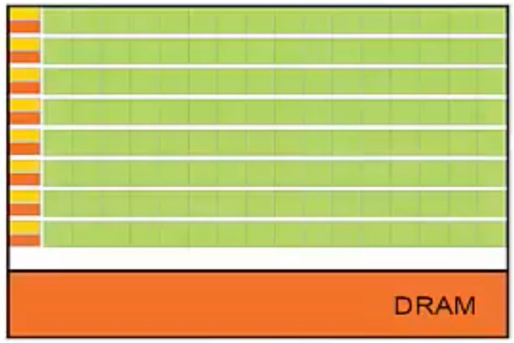
- **Tensor Operations**:
  - GPUs handle **tensor operations** like addition, multiplication, or softmax, especially in neural networks with millions of parameters.
- **Inference Optimization (Quantization and TensorRT)**:
  - During inference, **GPU-specific libraries** like **TensorRT** can accelerate computation by running quantized models efficiently.
- **Parallel Computation Across Batches**:
  - GPUs enable **batch-level parallelism** during both training and inference by processing multiple samples simultaneously.

Summary

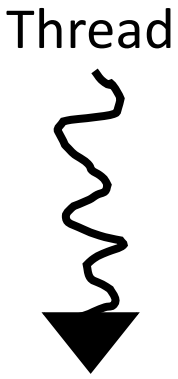
Operation	Handled by CPU	Handled by GPU
Data loading and preprocessing	✅	
Model initialization and control	✅	
Forward pass (inference)		✅
Backward pass (gradient computation)		✅
Weight updates	Sometimes	✅
Tensor operations		✅
Gradient aggregation (multi-GPU)	✅	
Mixed precision coordination	✅	
Model saving/loading	✅	
Batch parallelism		✅

CUDA Threads

- Kernels execute as a **set of parallel threads**
- CUDA is designed to execute **thousands of threads**
- **CUDA Threads execute in a SIMD** (Single Instruction Multiple Data) fashion
  - NVIDIA calls this SIMT (Single Instruction Multiple Thread)
- **Threads are similar to data-parallel tasks**
  - Each thread performs the same operation on a subset of data
  - Threads execute independently
- **Threads do not execute at the same rate**
  - Different paths are taken in if/else statements
  - Different number of iterations in a loop, etc.



GPU



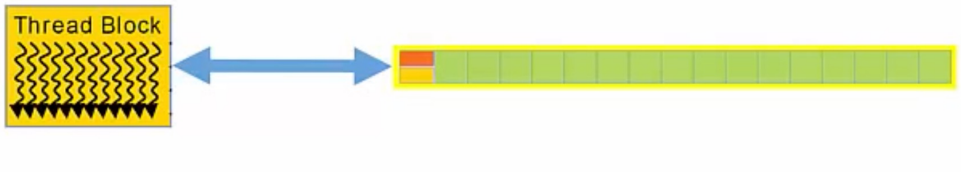
Threads Hierarchy

Threads

- Kernels execute as a set of Threads

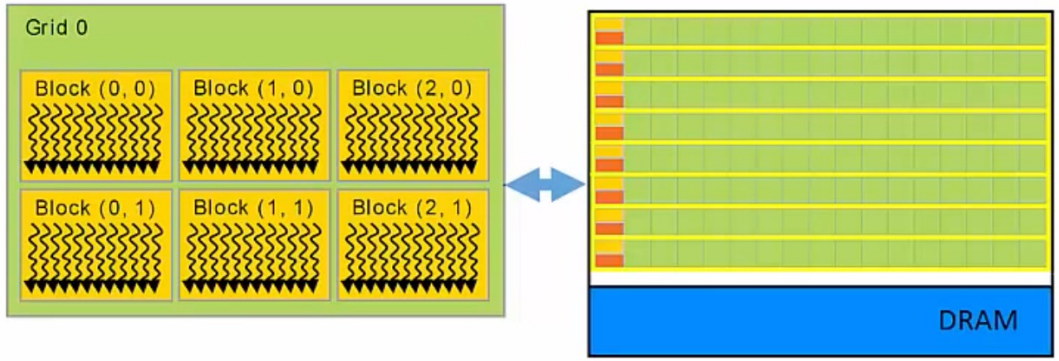
Blocks

- Threads are groups into blocks



Grid

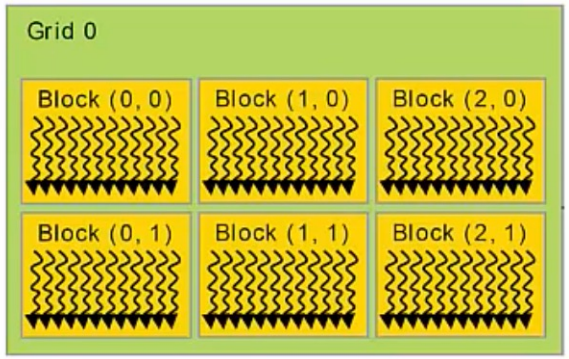
- Blocks are grouped into Grids
- **Each Kernel launch creates a single Grid**



Thread $\in$ Blocks $\in$ Grids

Dimensions of Gids and Blocks

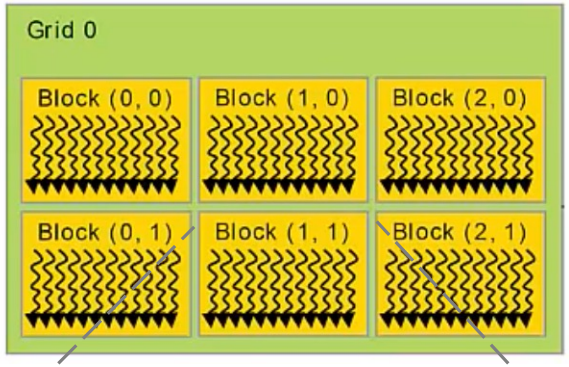
Grid dimension
- Block structure of each Grid
- 1D, 2D, or 3D



Grid dimension: 3 x 2
=> 6 block

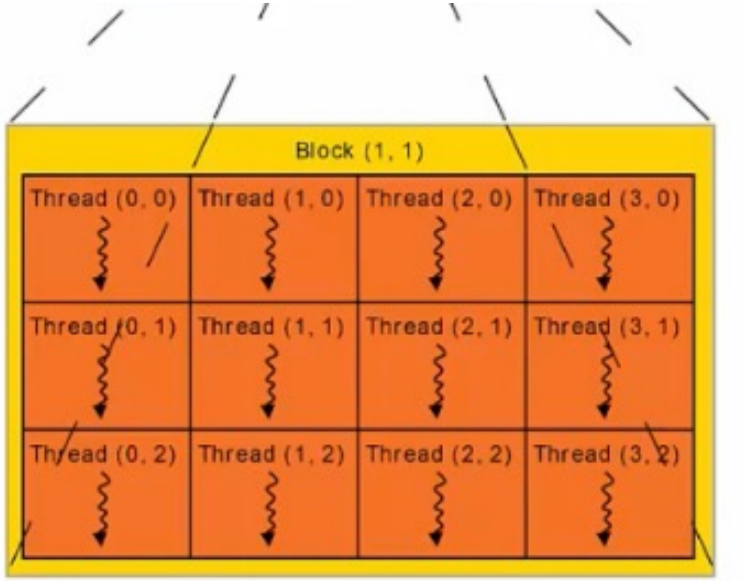
Dimensions of Gids and Blocks

Grid dimension
- Block structure of each Grid
- 1D, 2D, or 3D



Grid dimension: 3 x 2
=> 6 block

Grid dimension
- Thread structure of each Block
- 1D, 2D, or 3D



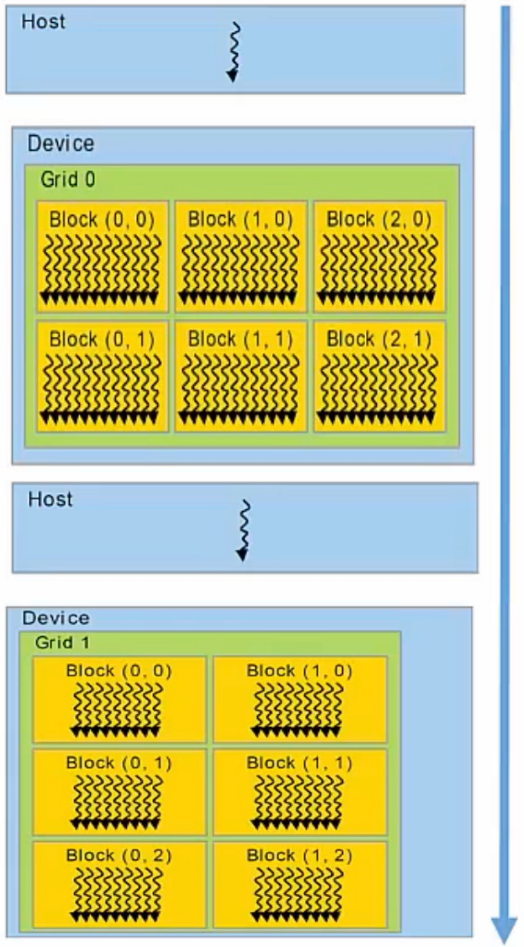
Block Dimension: 4x3
⇒ 4x3 = 12 Threads/Block
⇒ 6 Blocks x 12 Threads/Block
⇒ 72 Total Threads in Grid

Program Flow

- Host Code
  - Serial
  - Runs on CPU
  - Launches Kernels

- Device Code
  - Parallel
  - Runs on GPU

```
int main( void ) { // Host Code
  // Do sequential stuff

  // Launch Kernel
  kernel_0<<< grid_sz0,blk_sz0 >>>(...);

  // Do more sequential stuff

  // Launch Kernel
  kernel_1<<< grid_sz1,blk_sz1 >>>(...);

} return 0;
```



Host code does not wait for the GPU kernel to finish unless explicitly told so

Kernel Launch

```
// Block and Grid dimensions
dim3 grid_size(x, y, z)
dim3 block_size(x, y, z)
```

Configuration Parameters

<<<grid_size,block_size>>>

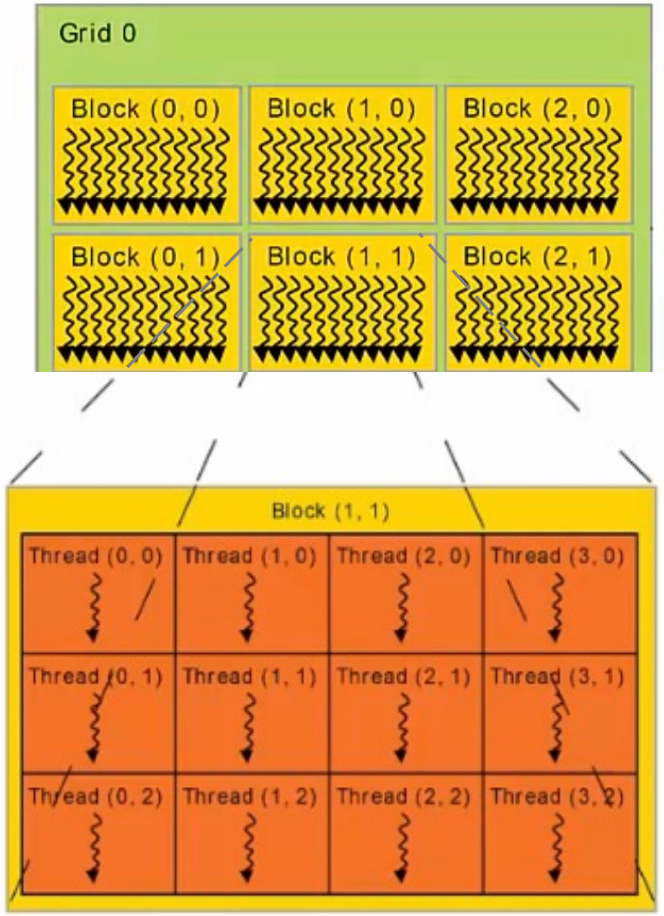
- dim3 is a CUDA data structure
- Default values are (1,1,1)

```
// Launch Kernel
KernelName<<< grid_size, block_size >>>( . . . );
```

Kernel Launch Configuration Example

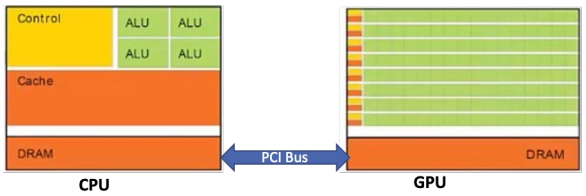
```
// Block and Grid dimensions
// a.k.a. Configuration Parameters
dim3 grid_size(3,2);
dim3 block_size(4,3);
```

```
// Launch Kernel
kernelName<<< grid_size, block_size >>>( ... );
```

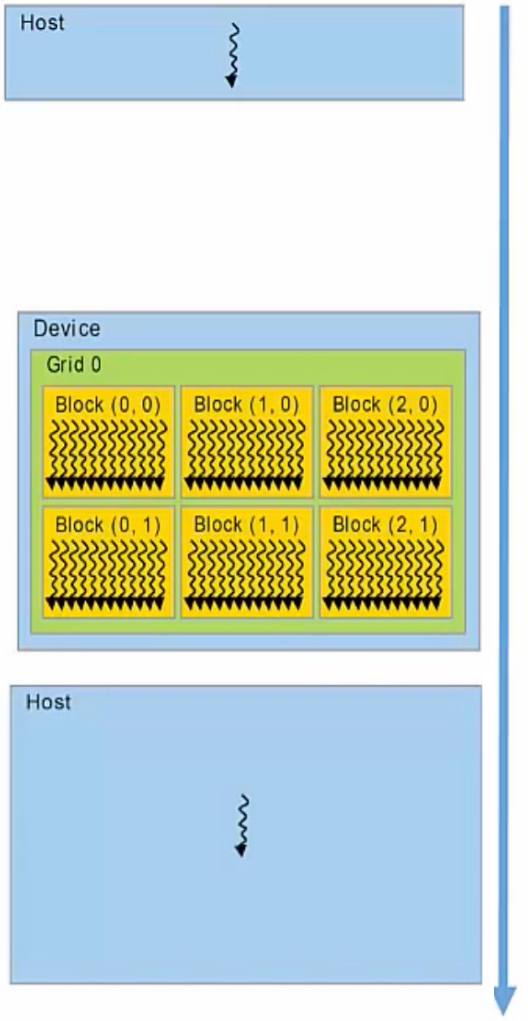


Closer look at Program Flow

- Host Code
  - Do sequential staff
  - Prepare for Kernel Launch
- Allocate Memory on Device
- Copy Data Host->Device
  - Copy data from CPU to GPU
- Launch Kernel
  - Execute Threads on the GPU in Parallel
- Copy Data Device->Host

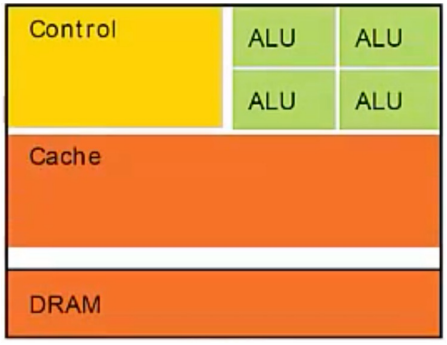



```
int main( void ) { // Host Code
  // Do sequential stuff

  // Allocate memory on the device
  cudaMalloc(...);

  // Copy data from Host to Device
  cudaMemcpy(...);

  // Launch Kernel
  kernel_0<<< grid_sz0,blk_sz0 >>>(...);

  // Copy data from Device to Host
  cudaMemcpy(...);

  // Do sequential stuff
  // ...

  return 0;
}
```

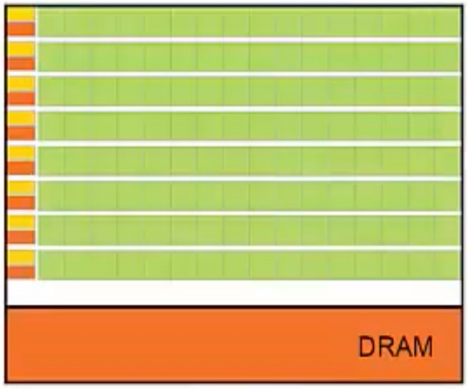


Allocation Device Memory

- Similar to allocation of memory in C
  - malloc(. . .);
- De-Allocate memory in C
  - free(. . .);
- Allocate memory in CUDA
  - cudaMalloc(LOCATION, SIZE);
  - 1st Argument:
    - Memory location on Device to allocate memory
    - An address in the GPU's memory
  - 2nd Argument:
    - Number of bytes to allocate
- De-Allocate memory in CUDA
  - cudaFree(. . .);



CPU



GPU

Copy Data Host <-> Device

```
cudaMemcpy( dst, src, numBytes, direction );
```

- numBytes = N*sizeof(type)
- direction
  - cudaMemcpyHostToDevice
  - cudaMemcpyDeviceToHost

Example

```
int main( void ) {

    // Declare variables
    int *h_c, *d_c;

    // Allocate memory on the device
    cudaMalloc( (void**)&d_c, sizeof(int) );

    // Allocate memory on the device
    cudaMemcpy(d_c, h_c, sizeof(int), cudaMemcpyHostToDevice );

    // Configuration Parameters
    dim3 grid_size(1);    dim3 block_size(1);

    // Launch the Kernel
    kernel<<<grid_size, block_size>>>(...);

    // Copy data back to host
    cudaMemcpy( h_c, d_c, sizeof(int), cudaMemcpyDeviceToHost );

    // De-allocate memory
    cudaFree( d_c );  free( h_c );
    return 0;
}
```

// Declare variable

Convention

- Variables that live on Host
  - h_
- Variables that live on Device
  - d_

Example

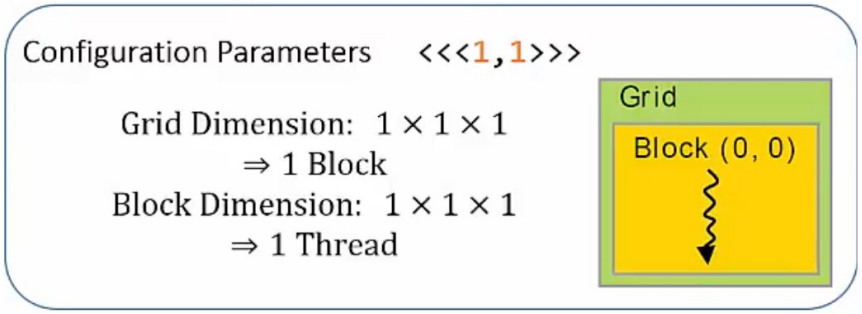
```
int main( void ) {

    // Declare variables
    int *h_c, *d_c;

    // Allocate memory on the device
    cudaMalloc( (void**)&d_c, sizeof(int) );

    // Allocate memory on the device
    cudaMemcpy(d_c, h_c, sizeof(int), cudaMemcpyHostToDevice );

    // Configuration Parameters
    dim3 grid_size(1);    dim3 block_size(1);

    // Launch the Kernel
    kernel<<<grid_size, block_size>>>(...);

    // Copy data back to host
    cudaMemcpy( h_c, d_c, sizeof(int), cudaMemcpyDeviceToHost );

    // De-allocate memory
    cudaFree( d_c );  free( h_c );
    return 0;
}
```

// Allocate memory on device.

cudaMalloc(Location of Memory on Device, Amount of Memory)

Example

```
int main( void ) {

    // Declare variables
    int *h_c, *d_c;

    // Allocate memory on the device
    cudaMalloc( (void**)&d_c, sizeof(int) );

    // Allocate memory on the device
    cudaMemcpy(d_c, h_c, sizeof(int), cudaMemcpyHostToDevice );

    // Configuration Parameters
    dim3 grid_size(1);    dim3 block_size(1);

    // Launch the Kernel
    kernel<<<grid_size, block_size>>>(...);

    // Copy data back to host
    cudaMemcpy( h_c, d_c, sizeof(int), cudaMemcpyDeviceToHost );

    // De-allocate memory
    cudaFree( d_c );  free( h_c );
    return 0;
}
```



Kernel Definition

```
__global__ void kernel( int *d_out, int *d_in )
{
    // Perform this operation for every thread
    d_out[0] = d_in[0];
}
```

__global__ is a "Declaration Specifier" that alerts the compiler that a function should be compiled to run on device.

Kernels must return type void
⇒ Variables operated on in the kernel need to be passed by reference

Any results that the kernel computes are stored in the device memory.

To manipulate data:

C uses "pass-by-value"

- Functions receive copies of their arguments
- The actual parameters to the function will not be modified.
- We simulate "pass-by-reference".
  - Pass the address of the variable as parameter to the kernel.

How to distinguish Threads from one another?

Thread Index

- Each Thread has its own thread index
  - Accessible within a Kernel through the built in threadIdx variable
- Thread Blocks can have as many as 3-dimensions, therefore there is a corresponding index for each dimension:

```
threadIdx.x
threadIdx.y
threadIdx.z
```

```
// Configuration Parameters
dim3 grid_size(1);
dim3 block_size(N);
```

Grid Dimension: $1 \times 1 \times 1$
$\Rightarrow$ 1 Block

Block Dimension: $N \times 1 \times 1$
$\Rightarrow N$ Threads

Parallelize for loop example

CPU Program

```
// Function Definition
void increment_cpu( int *a, int N)
{
    for (int i=0; i<N; i++)
        a[i] = a[i] + 1;
}

int main( void )
{
    int a[N] =  // ...

    // Call Function
    increment_cpu( a , N );
    // ...
    return 0;
}
```

CUDA Program

```
// Kernel Definition
__global__ void increment_gpu(int *a, int N)
{
    int i = threadIdx.x;
    if (i < N)
        a[i] = a[i] + 1;
}

int main( void )
{
    int h_a[N] =  // ...

    // Allocate arrays in Device memory
    int* d_a; cudaMalloc( (void**)&d_a, N * sizeof(int) );

    // Copy memory from Host to Device
    cudaMemcpy(d_a, h_a, N*sizeof(int), cudaMemcpyHostToDevice );

    // Block and Grid dimensions
    dim3 grid_size(1);   dim3 block_size(N);

    // Launch Kernel
    increment_gpu<<<grid_size, block_size>>>( d_a , N );
    // ...
    return 0;
}
```

Parallelize for loop example

CPU Program

```
// Function Definition
void increment_cpu( int *a, int N)
{
    for (int i=0; i<N; i++)
        a[i] = a[i] + 1;
}

int main( void )
{
    int a[N] =  // ...

    // Call Function
    increment_cpu( a , N );
    // ...
    return 0;
}
```

CUDA Program

```
// Kernel Definition
__global__ void increment_gpu(int *a, int N)
{
    int i = threadIdx.x;
    if (i < N)
        a[i] = a[i] + 1;
}

int main( void )
{
    int h_a[N] =  // ...

    // Allocate arrays in Device memory
    int* d_a; cudaMalloc( (void**)&d_a, N * sizeof(int) );

    // Copy memory from Host to Device
    cudaMemcpy(d_a, h_a, N*sizeof(int), cudaMemcpyHostToDevice );

    // Block and Grid dimensions
    dim3 grid_size(1);   dim3 block_size(N);

    // Launch Kernel
    increment_gpu<<<grid_size, block_size>>>( d_a , N );
    // ...
    return 0;
}
```

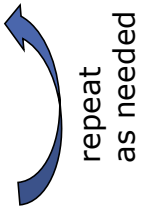
If (i < N)
Ensures that the kernel does not execute more Threads than the length of the array.

Traditional Program Structure in CUDA

- Function prototypes

```
float serialFunction(…);
__global__ void kernel(…);
```

- `main()`
  - 1) Allocate memory space on the device – `cudaMalloc(&d_in, bytes);`
  - 2) Transfer data from host to device – `cudaMemCpy(d_in, h_in, …);`
  - 3) Execution configuration setup: #blocks and #threads
  - 4) Kernel call – `kernel<<<execution configuration>>>(args…);`
  - 5) Transfer results from device to host – `cudaMemCpy(h_out, d_out, …);`



- Kernel – `__global__ void kernel(type args,…)`
  - Automatic variables transparently assigned to registers
  - Shared memory: `__shared__`
  - Intra-block synchronization: `__syncthreads();`

Slide credit: Hwu & Kirk

CUDA Programming Language

- Memory allocation

```
cudaMalloc((void**)&d_in, #bytes);
```

- Memory copy

```
cudaMemcpy(d_in, h_in, #bytes, cudaMemcpyHostToDevice);
```

- Kernel launch

```
kernel<<< #blocks, #threads >>>(args);
```

- Memory deallocation

```
cudaFree(d_in);
```

- Explicit synchronization

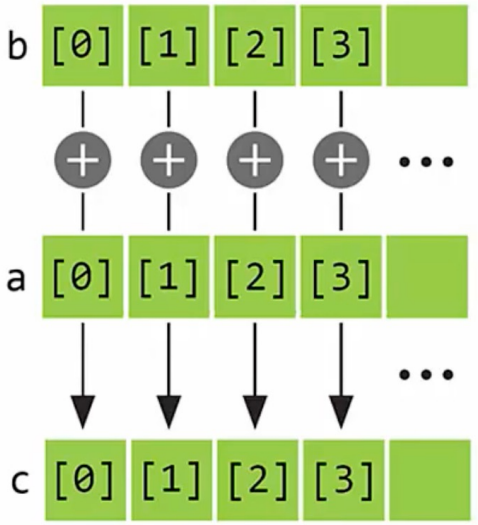
```
cudaDeviceSynchronize();
```

Vector Addition – A Very Parallel Problem

$$\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^N$$

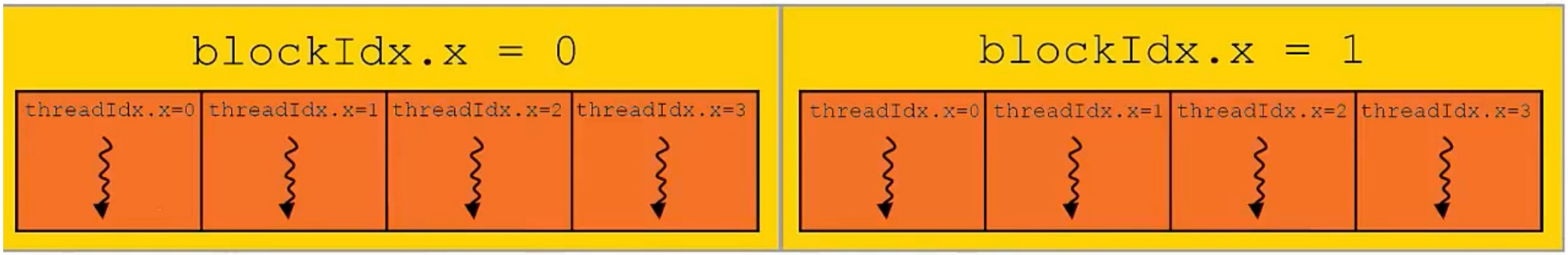
$$\boldsymbol{a} + \boldsymbol{b} = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{N-1} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{N-1} \end{bmatrix}$$

$$= \begin{bmatrix} a_0 + b_0 \\ a_1 + b_1 \\ a_2 + b_2 \\ a_3 + b_3 \\ \vdots \\ a_{N-1} + b_{N-1} \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_{N-1} \end{bmatrix} = \boldsymbol{c} =$$

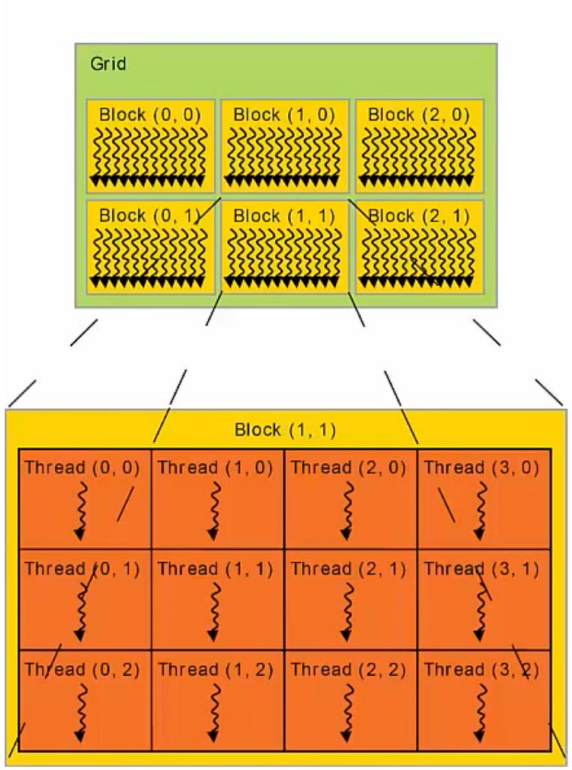


Thread Index

- Each Thread has its own unique thread index.
  - Accessible within a Kernel through the built in `threadIdx` variable.



Built-in Variables



Dimension of a Grid

```
dim3 gridDim;
int gridDim.x;
int gridDim.y;
int gridDim.z;
```

Index of a Block

```
dim3 blockIdx;
int blockIdx.x;
int blockIdx.y;
int blockIdx.z;
```

Dimension of a Block

```
dim3 blockDim;
int blockDim.x;
int blockDim.y;
int blockDim.z;
```

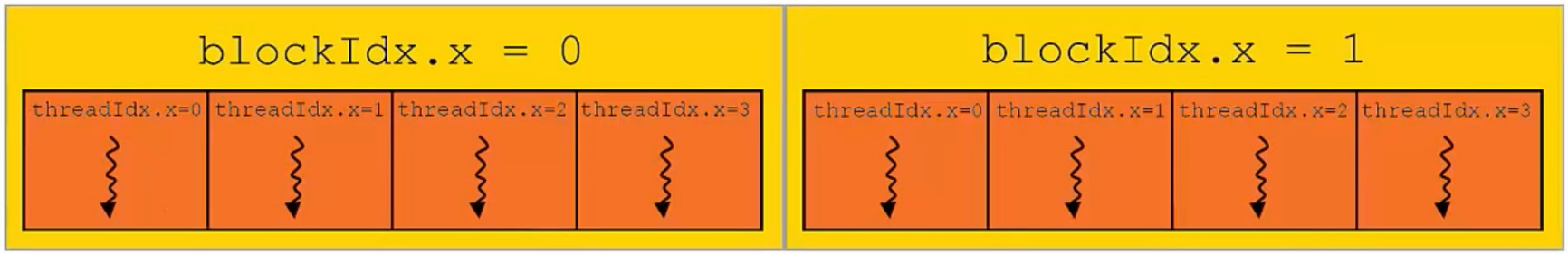
Index of a Thread

```
dim3 threadIdx;
int threadIdx.x;
int threadIdx.y;
int threadIdx.z;
```

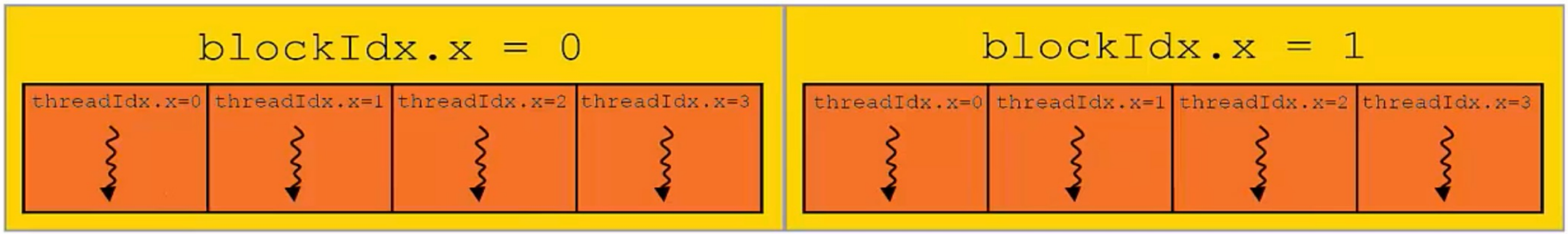
Indexing Within Grid

- `threadIdx` is only unique within its own Thread Block
- To determine the unique Grid index of a Thread:

```
i = threadIdx.x + blockIdx.x * blockDim.x;
```



Indexing Within Grid




```
i = threadIdx.x + blockIdx.x * blockDim.x;
```

i	threadIdx.x	blockIdx.x * blockDim.x
0	0	0 * 4 = 0
1	1	0 * 4 = 0
2	2	0 * 4 = 0
3	3	0 * 4 = 0
4	0	1 * 4 = 4
5	1	1 * 4 = 4
6	2	1 * 4 = 4
7	3	1 * 4 = 4

Examples

```
// Launch Kernel
kernel<<< 3, 4 >>>(a);
```

```
__global__ void kernel( int* a ) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    a[i] = blockDim.x;
}
```

```
__global__ void kernel( int* a ) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    a[i] = threadIdx.x;
}
```

```
__global__ void kernel( int* a ) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    a[i] = blockIdx.x;
}
```

```
__global__ void kernel( int* a ) {
      int i = threadIdx.x + blockIdx.x * blockDim.x;
      a[i] = i;
}
```

Examples

```
// Launch Kernel
kernel<<< 3, 4 >>>(a);
```

`__global__ void kernel( int* a ) {`
`  int i = threadIdx.x + blockIdx.x * blockDim.x;`
`  a[i] = blockDim.x;`
`}`
	a: 4 4 4 4 4 4 4 4 4 4 4 4

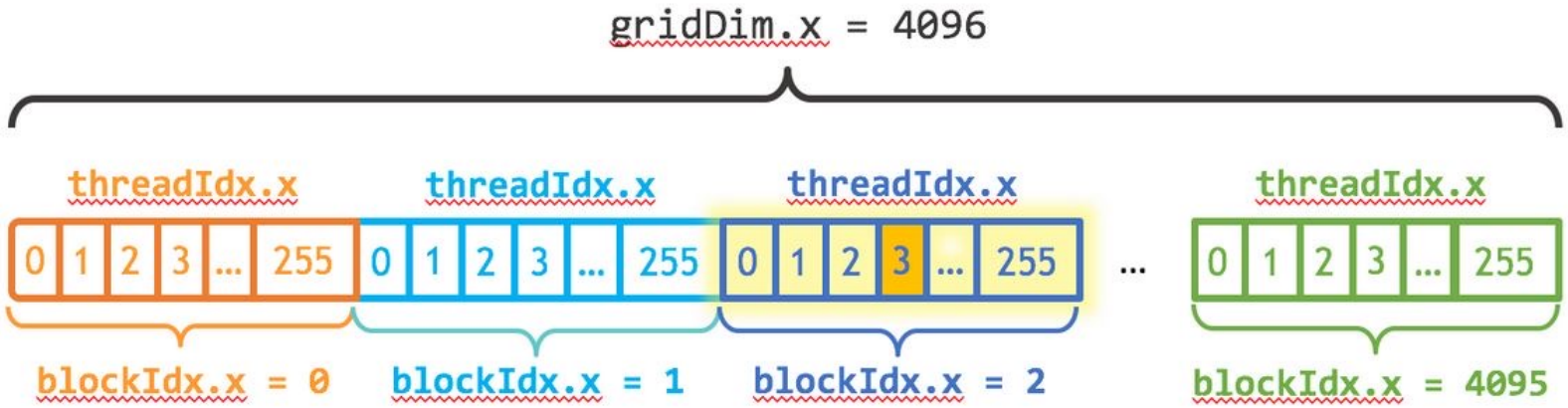
`__global__ void kernel( int* a ) {`
`  int i = threadIdx.x + blockIdx.x * blockDim.x;`
`  a[i] = threadIdx.x;`
`}`
	a: 0 1 2 3 0 1 2 3 0 1 2 3

`__global__ void kernel( int* a ) {`
`  int i = threadIdx.x + blockIdx.x * blockDim.x;`
`  a[i] = blockIdx.x;`
`}`
	a: 0 0 0 0 1 1 1 1 2 2 2 2

`__global__ void kernel( int* a ) {`
`  int i = threadIdx.x + blockIdx.x * blockDim.x;`
`  a[i] = i;`
`}`
	a: 0 1 2 3 4 5 6 7 8 9 10 11 12

Summary: CUDA – Grid Block and Thread

- One Grid per CUDA Kernel
- Multiple Blocks per Grid
- Multiple Threads per Block



From: NVIDIA

Indexing and Memory Access

- Images are 2D data structures
  - height x width
  - Image[j][i], where 0 ≤ j < height, and 0 ≤ i < width

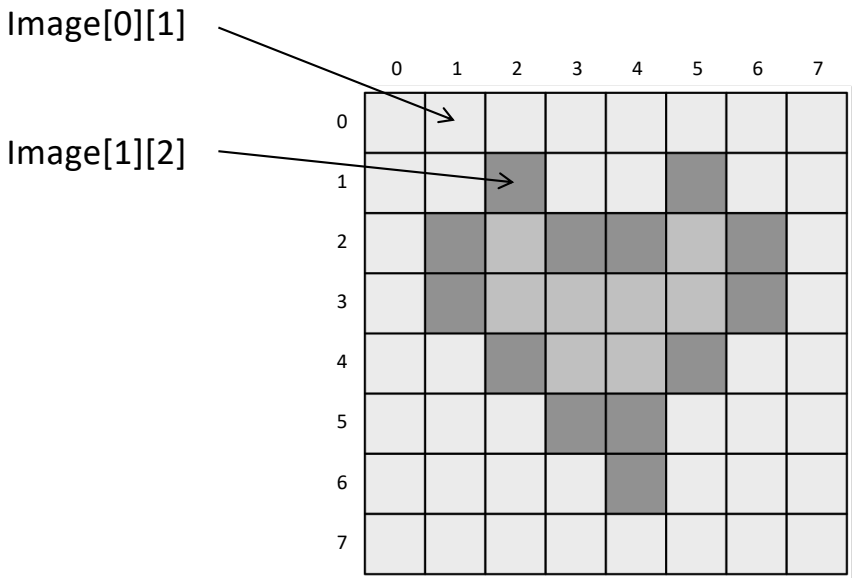
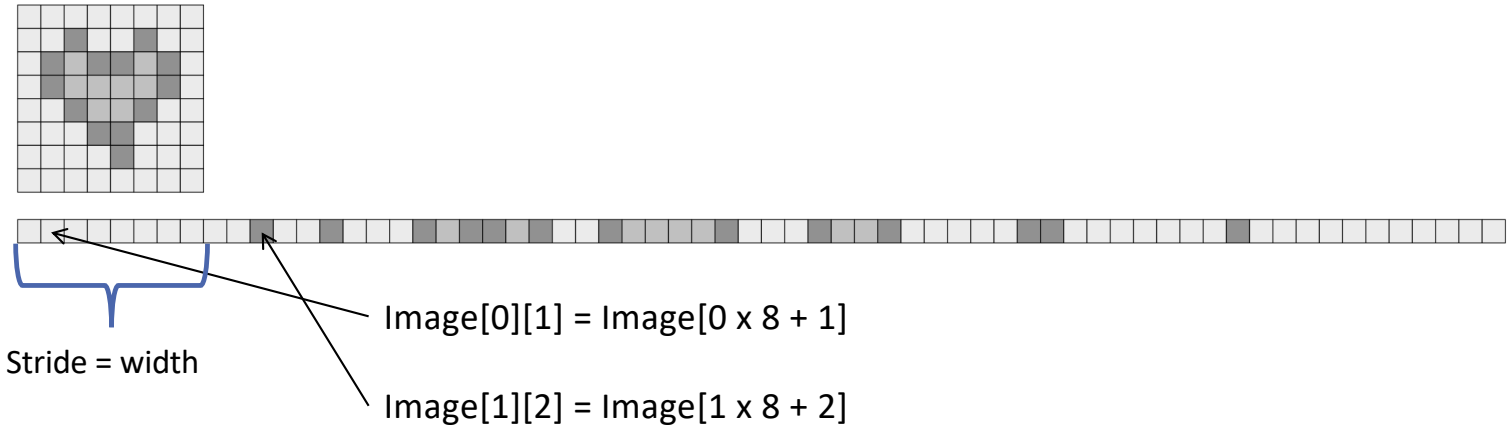


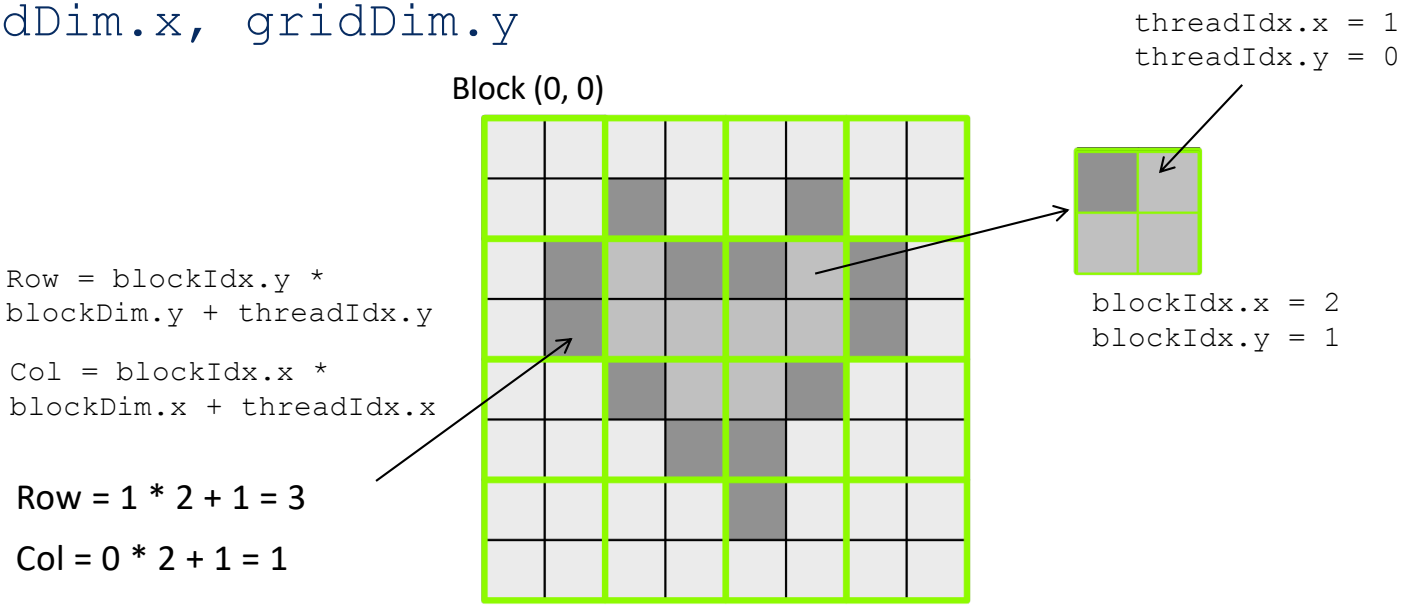
Image Layout in Memory

- Row-major layout
- Image[j][i] = Image[j x width + i]



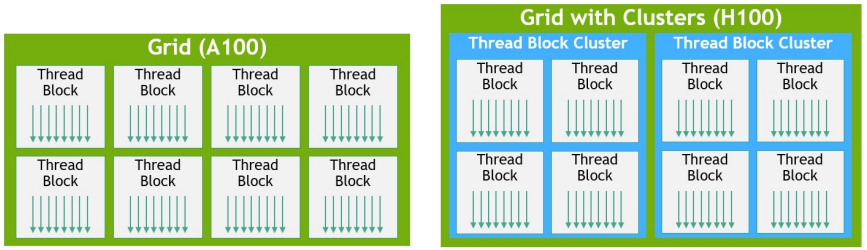
Indexing and Memory Access: 2D Grid

- 2D blocks
  - `gridDim.x, gridDim.y`



NVIDIA H100: Thread Block Clusters

- GPUs grow beyond 100 GPU cores (SMs): a new level in the software hierarchy can improve execution efficiency
  - Programmatic control of locality at a granularity larger than a single thread block on a single SM
- Thread blocks in the same cluster can synchronize and exchange data
- Thread blocks in the same cluster are guaranteed to be concurrently scheduled
  - Thread blocks in the same cluster run on the SMs within a GPU Processing Cluster (GPC)



https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

CUDA Memory Model

Thread Memory Correspondence

Threads ⇔ Local Memory (and Registers)

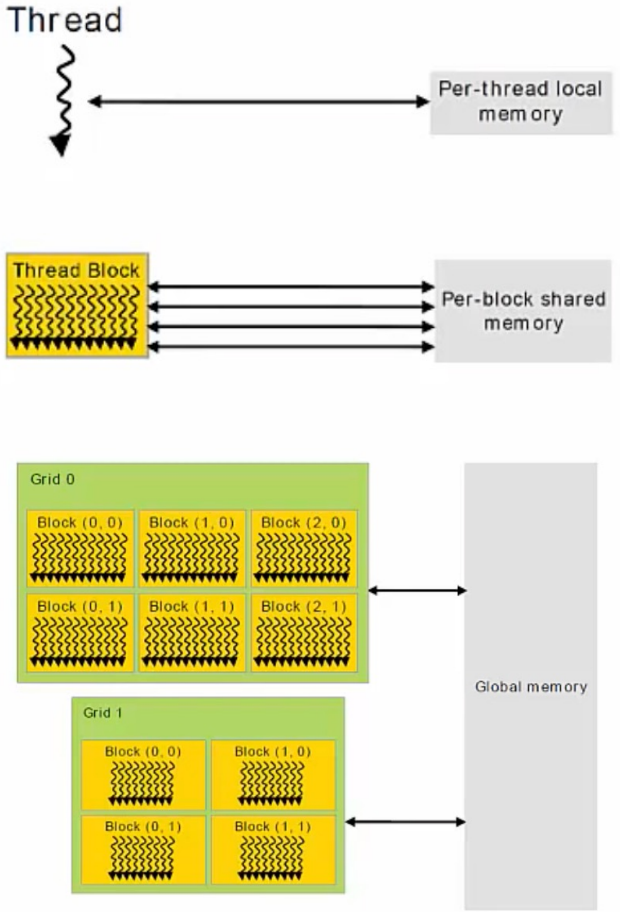
- Scope: Private to its corresponding Thread
- Lifetime: Thread

Blocks ⇔ Shared Memory

- Scope: Every Thread in the Block has access
- Lifetime: Block

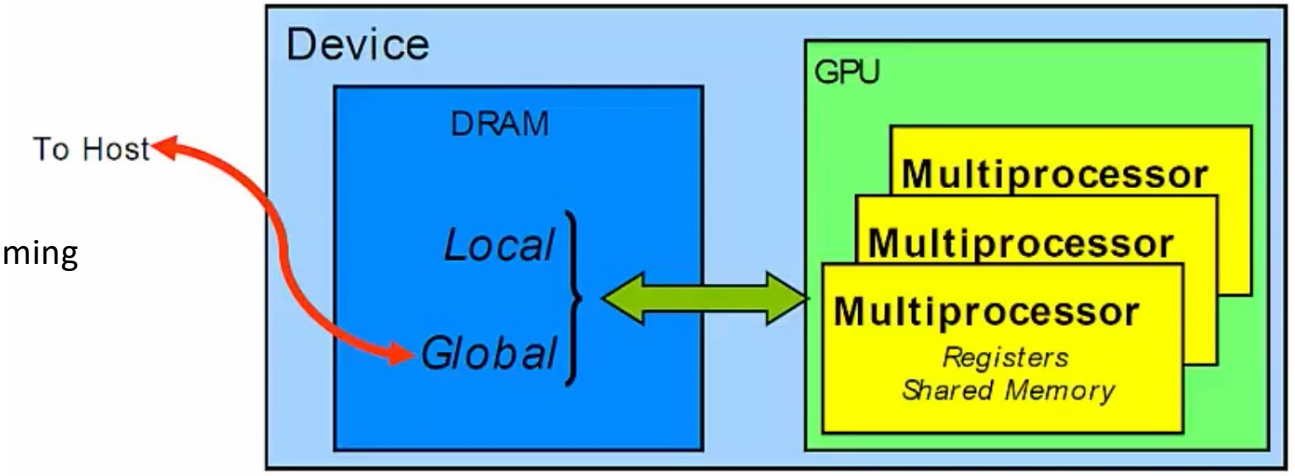
Grids ⇔ Global Memory

- Scope: Every Thread in all Grids have access
- Lifetime: Entire program in Host code – `main()`

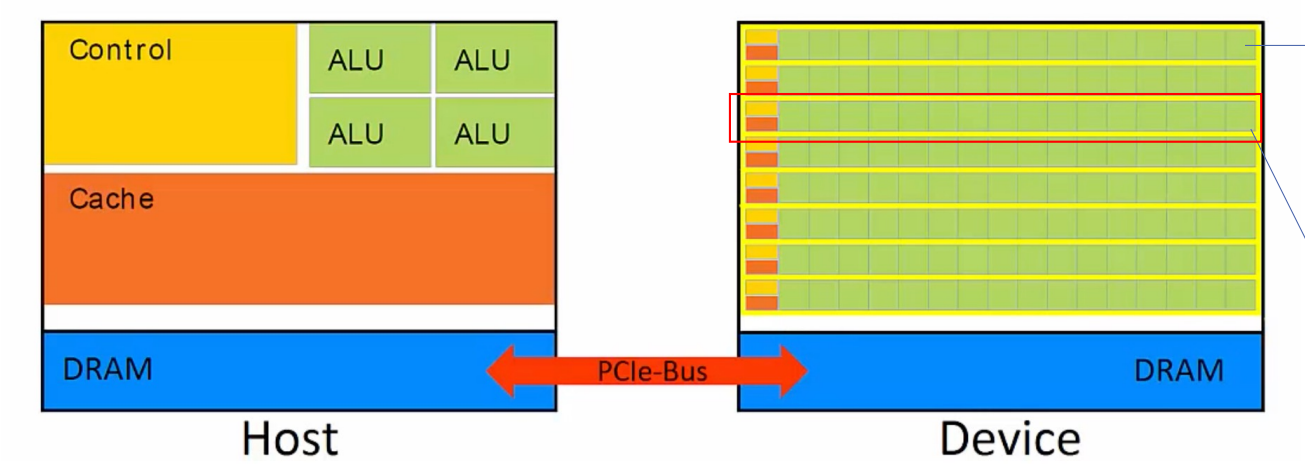


Memory Model

- Yellow rectangles represent SM: Streaming Multiprocessors
- Memory located in SMs is called:
  - **"On-chip"** Device memory
- Memory not in SM is called
  - **"Off-chip"** Device memory

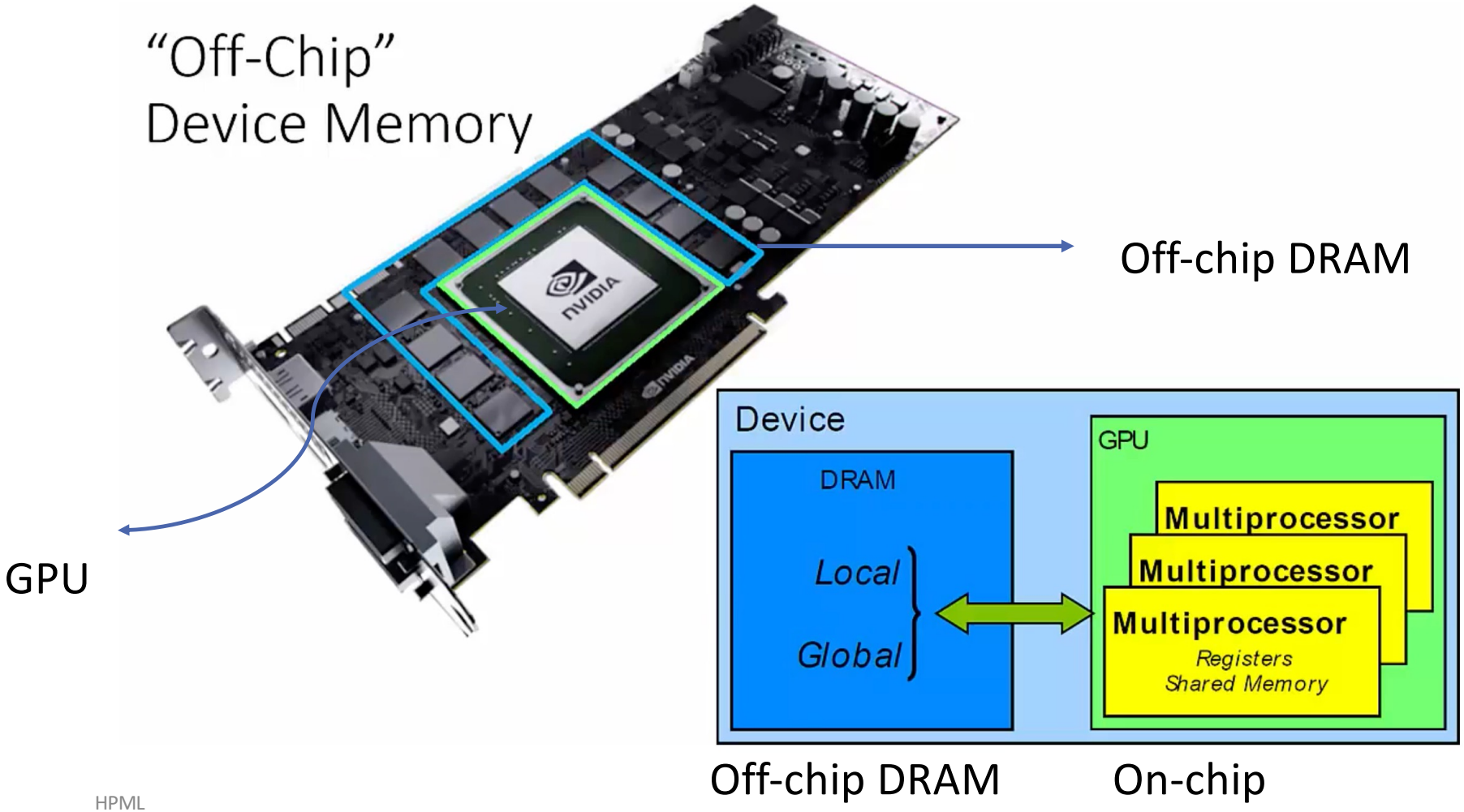


Term **local** refers to the scope of lifetime and not physical location

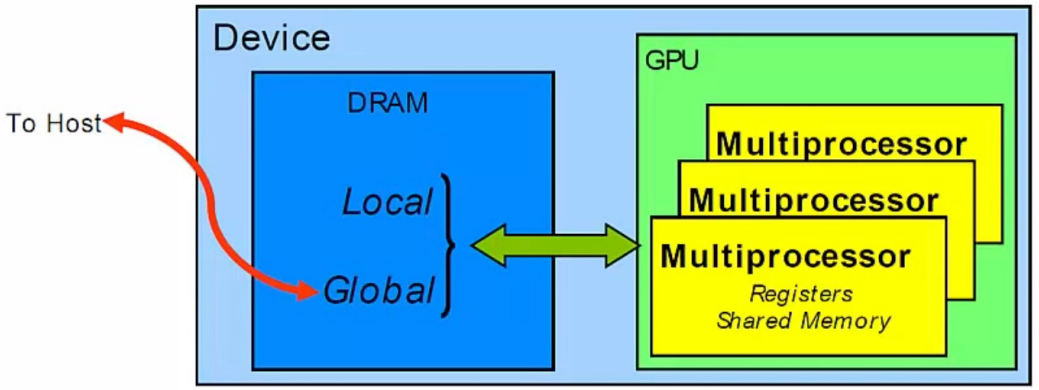


Streaming Multiprocessors

- An SM is a collection of processing units that work together to execute CUDA kernels. Each SM consists of several CUDA cores responsible for executing instructions in parallel.
- Each SM has "CUDA" cores, which are ALU units that can execute SIMD instructions
- The stream multiprocessor (SM) is a key computational grouping within a GPU, although "stream multiprocessor" is Nvidia's terminology. AMD would call them "compute units".
- "CUDA cores" would be called "shader units" or "stream processors" by AMD.


"Off-Chip"
Device Memory
Off-chip DRAM
GPU
Device
DRAM
Local
Global
GPU
Multiprocessor
Multiprocessor
Multiprocessor
Registers
Shared Memory
Off-chip DRAM
On-chip

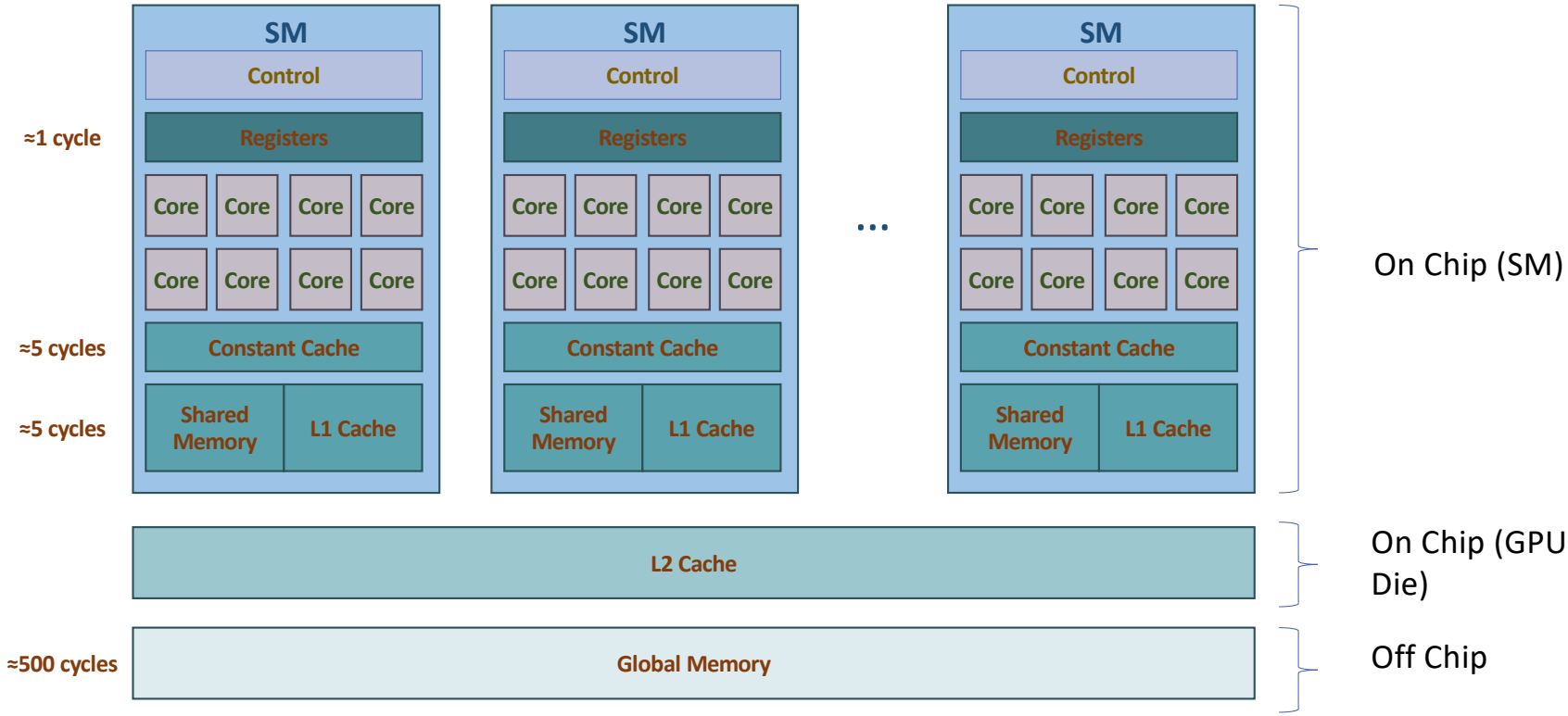
Memory Speed



- Relative speed of memory spaces: $\underbrace{\text{"Memory Space"}}_{\substack{\text{"Bandwidth"} \\ \text{"Latency"}}}$

$$\underbrace{\text{Registers}}_{\substack{\sim 8\text{TB/s} \\ \sim 1\text{clock}}} < \underbrace{\text{Shared}}_{\substack{\sim 1.5\text{TB/s} \\ \sim 32\text{clocks}}} \ll \underbrace{\text{Local} \approx \text{Global}}_{\substack{\sim 200\text{GB/s} \\ \sim 800\text{clocks}}} \ll \underbrace{\text{Host (PCIe)}}_{\sim 5\text{GB/s}}$$

Memory in the GPU Architecture

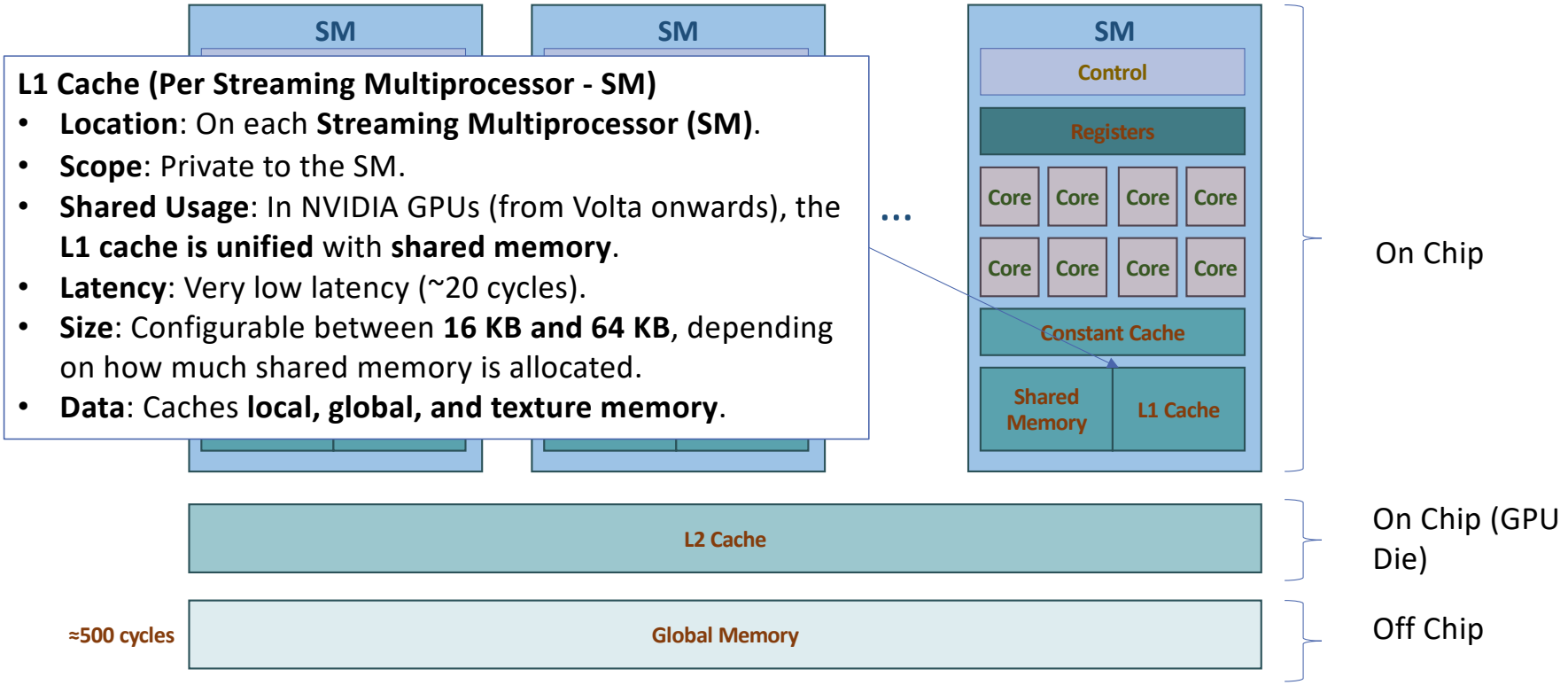


Slide credit: Izzat El Hajj

Memory in the GPU Architecture

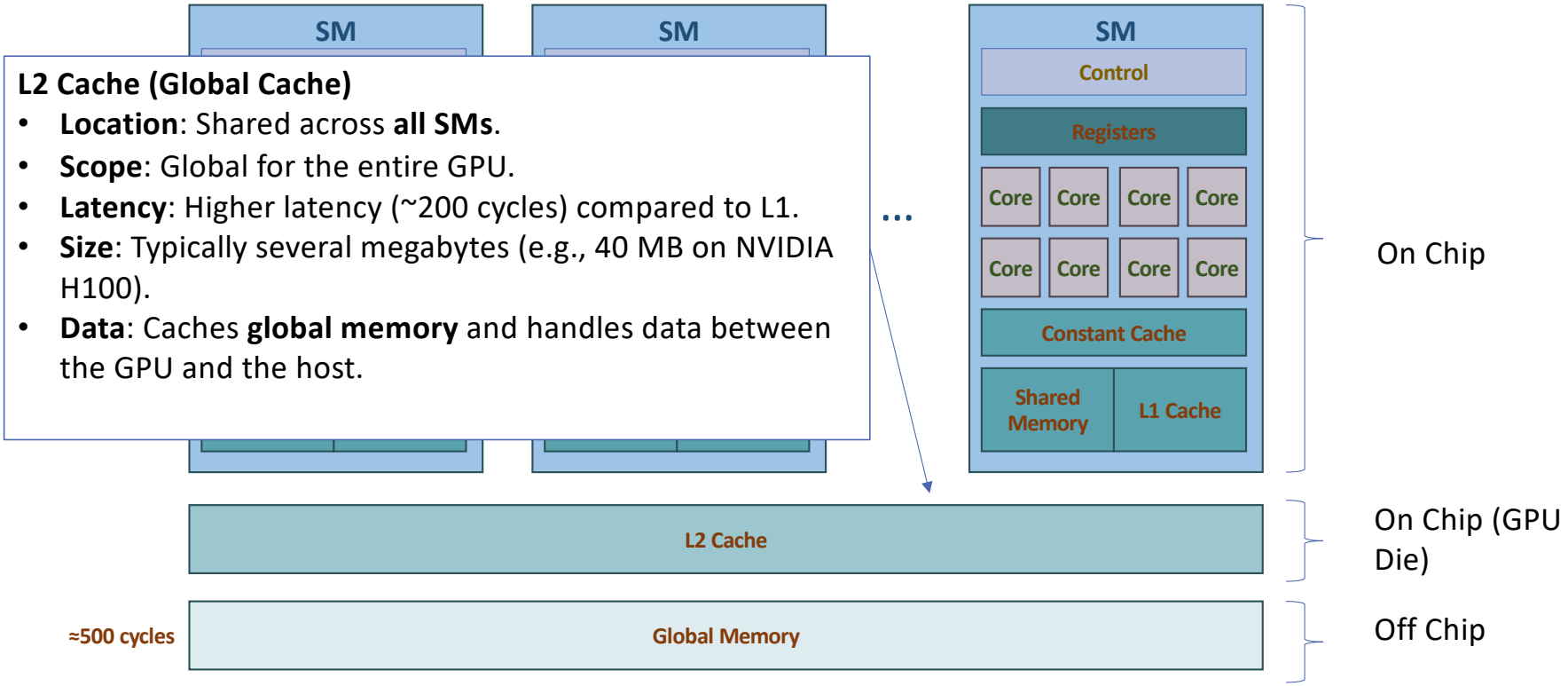
L1 Cache (Per Streaming Multiprocessor - SM)

- **Location**: On each **Streaming Multiprocessor (SM)**.
- **Scope**: Private to the SM.
- **Shared Usage**: In NVIDIA GPUs (from Volta onwards), the **L1 cache is unified** with **shared memory**.
- **Latency**: Very low latency (~20 cycles).
- **Size**: Configurable between **16 KB and 64 KB**, depending on how much shared memory is allocated.
- **Data**: Caches **local, global, and texture memory**.



Slide credit: Izzat El Hajj

Memory in the GPU Architecture



L2 Cache (Global Cache)

- **Location**: Shared across **all SMs**.
- **Scope**: Global for the entire GPU.
- **Latency**: Higher latency (~200 cycles) compared to L1.
- **Size**: Typically several megabytes (e.g., 40 MB on NVIDIA H100).
- **Data**: Caches **global memory** and handles data between the GPU and the host.

Slide credit: Izzat El Hajj

Illustration using the H100 Architecture

GH100 Full GPU with 144 SMs
Source: https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

H100 SM Architecture

- 4 Warp Schedulers & Dispatch Units per SM (32 threads/clock each).
- Register File: 16,384 × 32-bit registers per partition.
- Tensor Cores (4th Gen): Optimized for FP16, BF16, TF32, INT8, FP8 workloads.
- Execution Units: Support INT32, FP32, FP64, and special function units (SFUs).
- Memory System:
  - 256 KB combined L1 Data Cache / Shared Memory.
  - Dedicated Tensor Memory Accelerator (TMA) for efficient memory movement.
- Instruction Caches: L0 cache per partition, L1 instruction cache shared across SM.
- Texture Units: 4 per SM for graphics/compute operations.

L1 and L2 Caches

In CUDA GPUs, L1 and L2 caches play crucial roles in optimizing memory access and overall performance of the GPU. Here's an overview of their functions:

1. **L1 Cache**:
   - The L1 cache is a small, fast memory cache located in each Streaming Multiprocessor (SM) of a CUDA-capable GPU.
   - It is primarily used to cache local memory accesses by the threads within the same SM, including register spills and local array accesses.
   - L1 cache is very close to the CUDA cores in terms of the memory hierarchy, which means it offers low-latency access for frequently used data.
   - In some NVIDIA GPU architectures, the L1 cache is combined with shared memory, giving developers the flexibility to balance between shared memory and L1 cache depending on their application's needs.
2. **L2 Cache**:
   - The L2 cache is a larger, unified cache that is shared across all SMs in the GPU.
   - It serves as a cache for global memory accesses, helping to reduce the latency and bandwidth demand on the main memory.
   - Since the L2 cache is shared across all SMs, it can efficiently cache data that is accessed by multiple SMs, making it particularly useful for applications with data sharing or reuse across different parts of the GPU.
   - The L2 cache also plays a role in atomic operations and global synchronization, as it ensures consistency across the multiple SMs.

L1 and L2 Caches

1. **Improving Performance**:
   - Efficient use of these caches can significantly improve the performance of CUDA applications. For instance, optimizing memory access patterns to increase cache hit rates can lead to lower memory latency and higher throughput.
   - Utilizing the L1 cache effectively often involves optimizing local memory usage within an SM, such as minimizing register spills and managing local array accesses.
   - To leverage the L2 cache, it's important to have memory access patterns that allow for effective global memory caching, like coalesced memory accesses that align with cache line boundaries.
2. **Architectural Considerations**:
   - Different generations of NVIDIA GPUs have different cache architectures and sizes. For example, the Kepler, Maxwell, Pascal, Volta, Turing, and Ampere architectures all have variations in how L1 and L2 caches are implemented and utilized.
   - Understanding the specific cache architecture of the target GPU can help in fine-tuning the performance of CUDA applications.

Summary- Memory Hierarchy

Memory Level	Location	Access Speed	Scope	A100 (Example)	H100 (Example)	Primary Use
Registers	On-chip (SM)	Extremely Fast	Private to one thread	Thousands per SM	Thousands per SM	Store thread-local variables and intermediate calculation results.
L1 Cache / Shared Memory	On-chip (SM)	Very Fast	Shared by threads in a block (L1 is private)	192 KB per SM (Configurable)	256 KB per SM (Configurable)	**Shared Memory:**Inter-thread communication, block-local data reuse. **L1 Cache:** Low-latency access to global/local memory.
L2 Cache	On-chip (GPU Die)	Fast	Shared by all SMs	40 MB unified cache	50 MB unified cache	High-hit-rate buffer for data from main memory, reducing latency to HBM.
Global Memory (HBM/HBM2e/HBM3)	Off-chip (GPU Package)	Slowest	Accessible by all SMs	Up to 80 GB HBM2e (~2 TB/s)	Up to 80 GB HBM3 (~3.35 TB/s)	Main memory for the GPU; stores large datasets, model parameters (weights), and I/O data.

Global Memory (VRAM)

This is the largest and slowest part of the hierarchy, serving as the GPU's main working memory.

A100 (Ampere)	H100 (Hopper)
• **HBM2e** (High-Bandwidth Memory 2 extended) with up to **80 GB** capacity • Bandwidth of up to **2 TB/s**.	• Upgrades to **HBM3** memory, maintaining **80 GB** capacity • Significantly increasing the bandwidth to up to **3.35 TB/s**. • This massive increase in bandwidth is critical for feeding data quickly to the more powerful processing cores (Tensor Cores) • One of the key factors for the H100's performance leap in large-scale AI training.

L2 Cache

This large, unified cache sits between the SMs and the Global Memory (HBM). Its primary role is to reduce the latency of accessing global memory by caching frequently used data.

A100 (Ampere)	H100 (Hopper)	A larger L2 cache
• Features a **40 MB** unified L2 cache.	• Increases the size to **50 MB** and improves the partitioning and management of the cache.	• => higher chance of finding the data on-chip, reducing slow off-chip accesses and improving the efficiency of data s

Shared Memory and L1 Cache

- These on-chip memories reside within each Streaming Multiprocessor (SM) and are essential for cooperative thread execution.
- Both L1 cache and Shared Memory often share the same physical memory space on the SM and are configurable by the programmer.

Shared Memory	L1 Cache
• Programmatically managed memory used by threads within the same thread block to exchange data quickly.	• Automatically managed cache for local and global memory accesses, reducing latency.

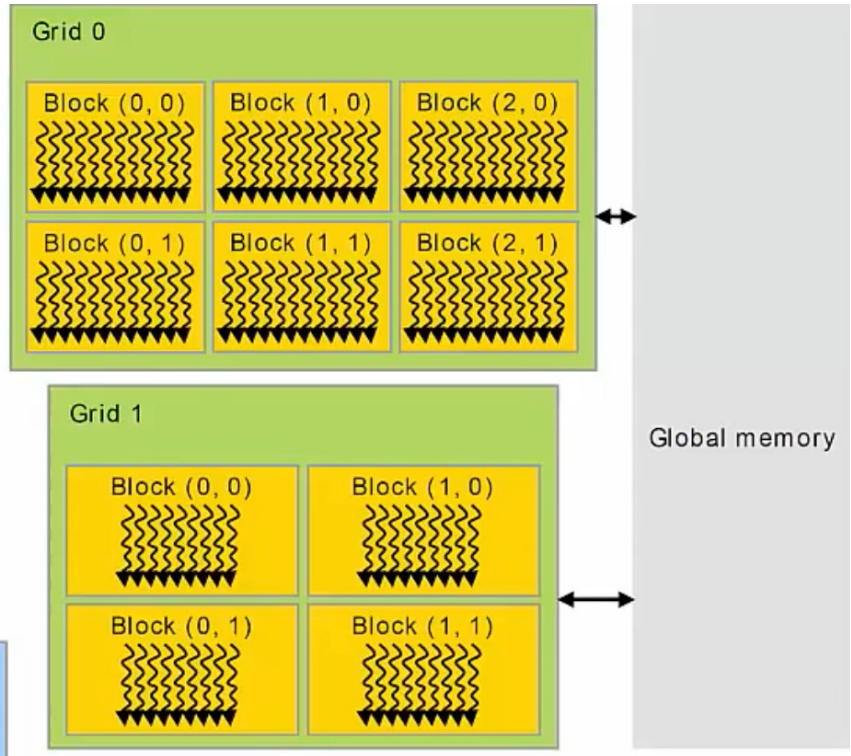
Programmatic Control of the Memory Hierarchy

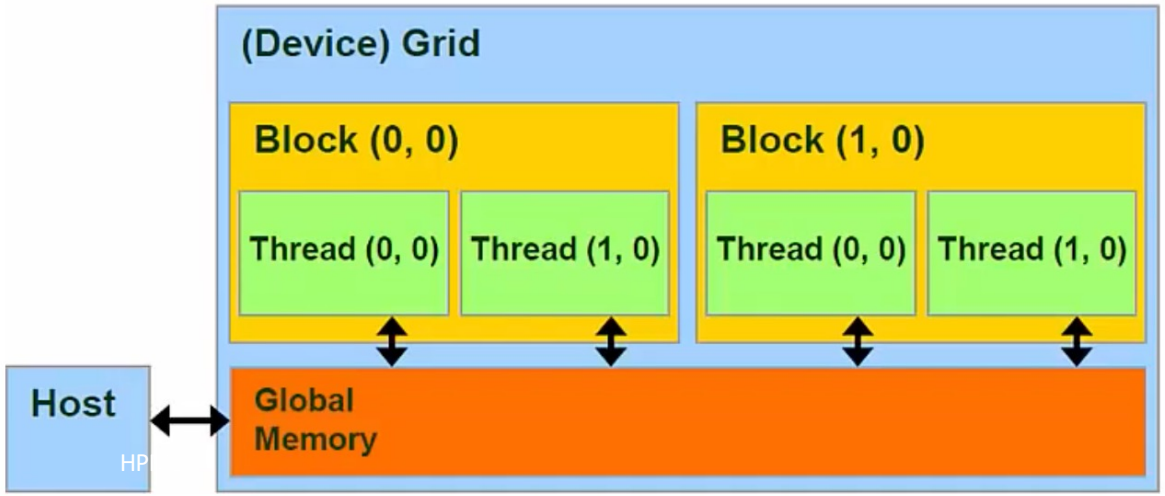
Memory Level	Control / Access Type	Programmatic Construct (CUDA C++)	Programmer Responsibility
Registers (Thread-Private)	**Implicit/Compiler-Managed**	Local variables in the kernel function.	Minimize local variable usage to avoid **register spilling** (which writes data to slow local memory).
Shared Memory (Block-Shared)	**Explicit/Full Control**	__shared__ keyword; extern __shared__ for dynamic allocation.	Manage allocation size, explicit **data transfer**, and **synchronization**(__syncthreads()).
L1/L2 Cache (SM/GPU-Shared)	**Coarse/Hint-Managed**	Compiler/driver flags, runtime APIs (cudaFuncSetCacheConfig), access patterns.	Design algorithms for **coalesced** access and **data locality** to maximize cache hits.
Global Memory (HBM/DRAM)	**Explicit Allocation & Transfer**	cudaMalloc, cudaMemcpy, __global__ memory accesses.	**Manage Host-to-Device data movement** (slowest operation) and optimize device-side access patterns.

Global Memory

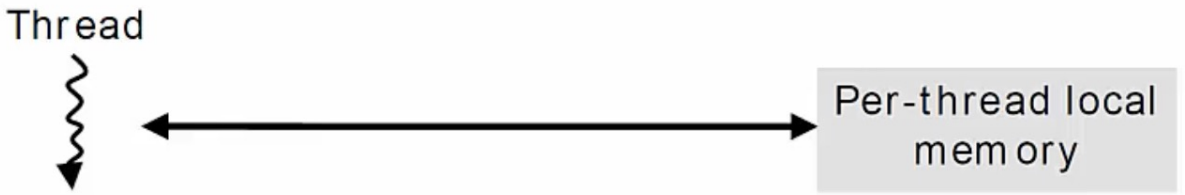
Accessed with

- `cudaMalloc()`
- `cudaMemset()`
- `cudaMemCopy()`
- `cudaFree()`

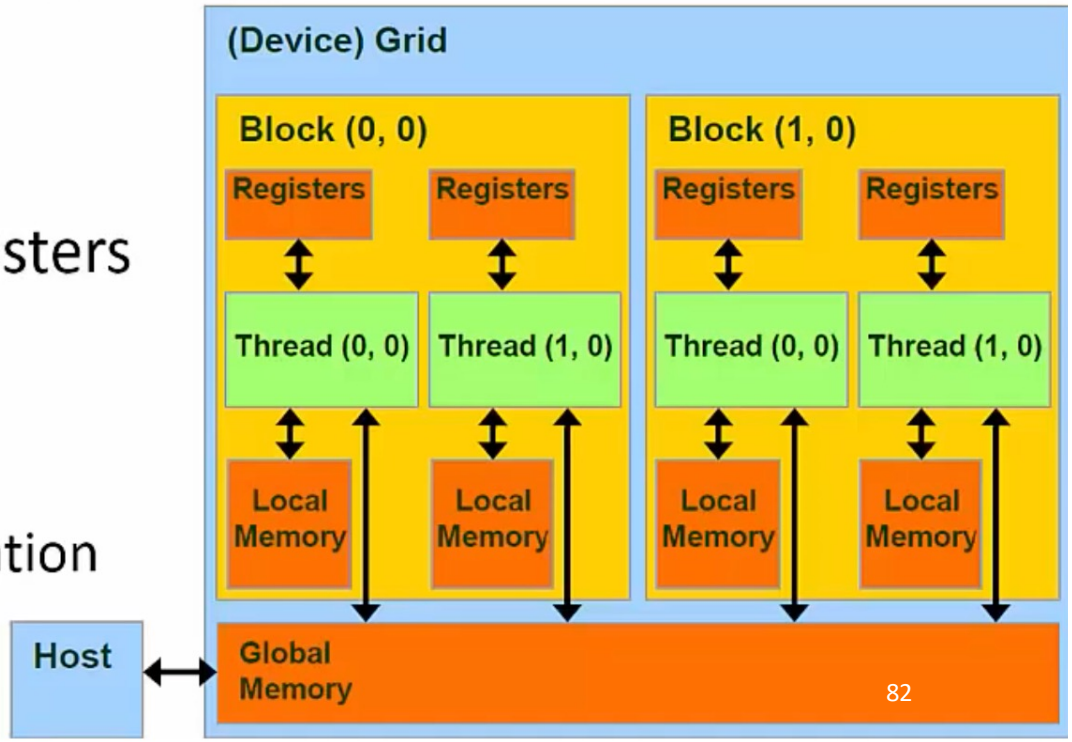




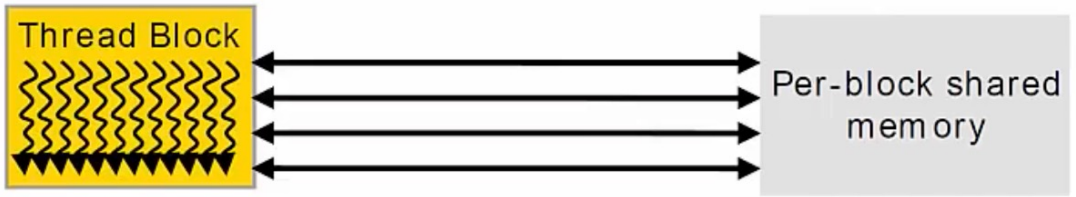
Registers and Local Memory



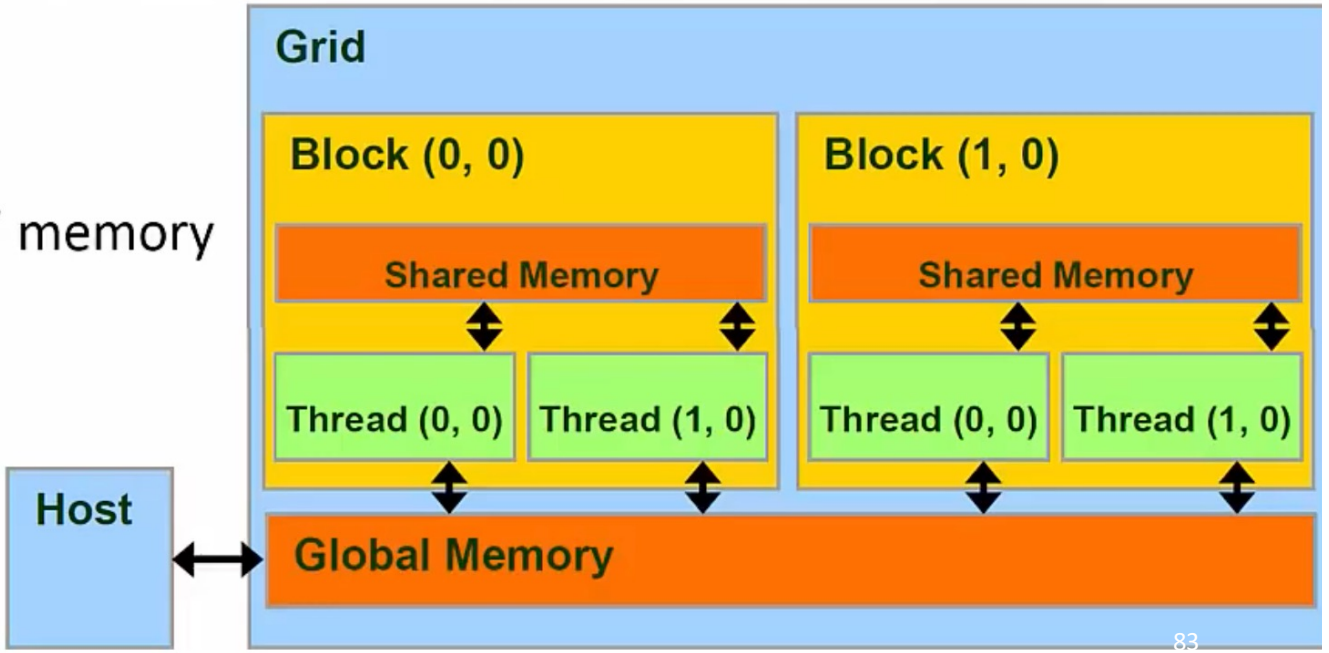
- Variables declared in a Kernel are stored in Registers
  - *On-Chip*
  - Fastest form of memory
- Arrays too large to fit into Registers spill over into Local memory
  - *Off-Chip*
  - Compiler controlled
  - "Local" refers to scope, not location
    - Local to each Thread



Shared Memory



- Allows Threads within a Block to communicate with each other
  - Use synchronization
- Very fast
  - Only Registers are faster
- Can use as “Scratch-pad” memory

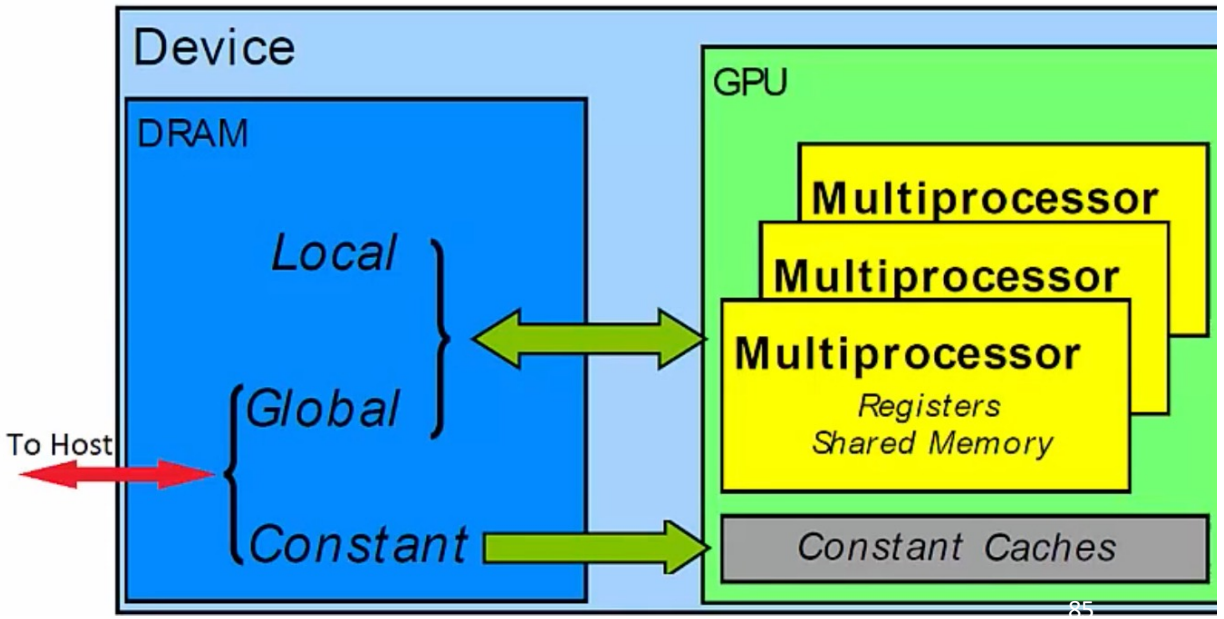


Using Shared Memory

```
__global__ void kernel( int* in, int N )
{
   int i  = threadIdx.x + blockDim.x * blockIdx.x;

   // Allocate a shared array
   __shared__ int shared_array[N];

   // each thread writes to one element of shared_array
   shared_array[i] = in[i];

   // Do more stuff
   // ...
}
```

Constant Memory

- Special Region of Device Memory
  - Used for data with unchanging contents throughout kernel execution
  - Read–Only from Kernel
- *Off Chip*
- Constant memory is aggressively cached into *On-Chip* memory



Memory Model

Registers & Local Memory

- Regular variables declared within a Kernel

Shared Memory

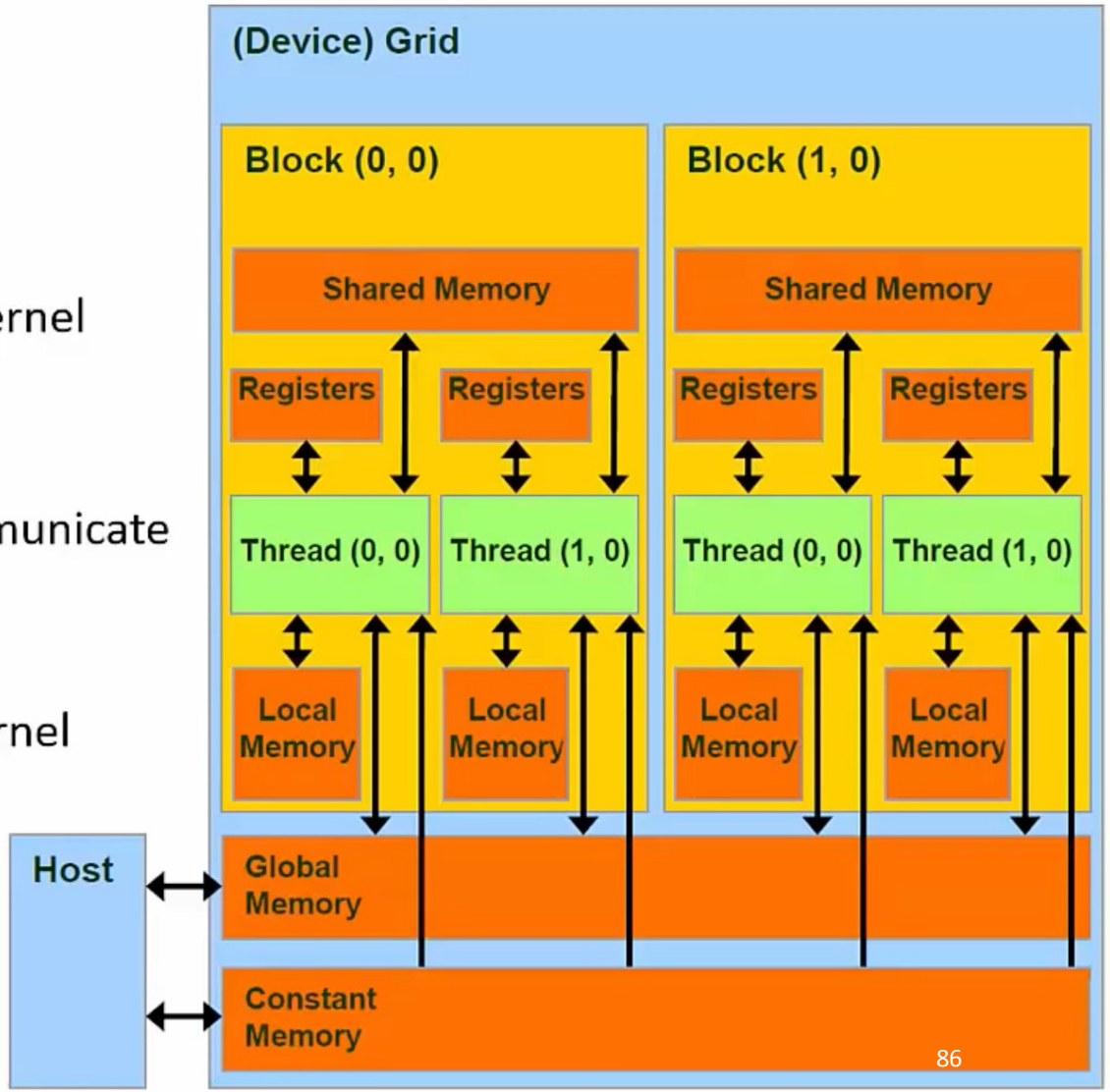
- Allows threads within a block to communicate

Constant

- Used for unchanging data through Kernel

Global Memory

- Stores data copied to and from Host



Registers Usage Best Practices

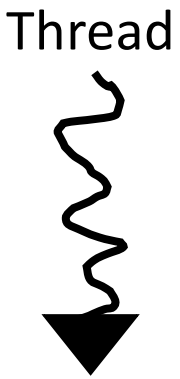
Here are some tips to help you manage register usage effectively:

1. **Understand Register Usage**: Each CUDA kernel invocation uses a certain number of registers. You can find out the number of registers used by your kernel by using the CUDA Compiler (nvcc) with the -Xptxas="-v" flag. This will give you information about the register usage per thread.
2. **Optimize Variable Usage**:
    - Minimize the number of local variables.
    - Use simpler data types when possible (e.g., float instead of double).
    - Reuse variables instead of declaring new ones for different purposes.
3. **Inline Functions**: If you use device functions, consider inlining them to reduce register usage. This can be done using the __forceinline__ qualifier.
4. **Loop Unrolling**: Sometimes, unrolling loops can reduce the number of registers used, but this could also increase register usage in other cases. You need to test and profile to see the effect.
5. **Limit Thread Usage**: The more threads you have per block, the fewer registers are available per thread. If you're hitting register limits, consider using fewer threads per block.
6. **Compiler Optimization Flags**: Use compiler optimization flags to control register usage. For example, --maxrregcount=N limits the number of registers that nvcc allows for each kernel.
7. **Shared Memory**: Consider using shared memory for some variables that do not need to be in registers. However, this might impact performance due to the slower access times compared to registers.
8. **Profiling and Optimization Tools**: Utilize CUDA profiling tools like nvprof or NVIDIA Nsight to understand how your kernel uses registers and to identify bottlenecks. These tools can provide insights that guide your optimization efforts.
9. **Architecture Considerations**: Different GPU architectures have different numbers of registers. Keep in mind the target GPU architecture when optimizing register usage.
10. **Manual Register Allocation**: In some advanced cases, you can manually manage registers through assembly-level programming using PTX, but this requires deep knowledge of GPU architecture and should be done with caution.

CUDA Thread Synchronization

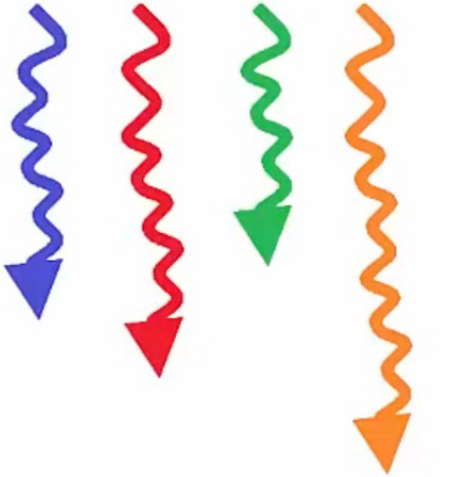
Threads Execute in Parallel

- Threads can read a result before another thread writes to that address
  - Race condition



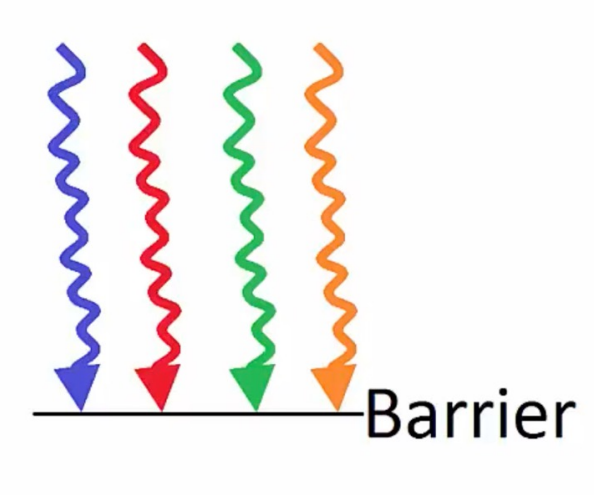
Thread Synchornization via Explicit Barrier

- Threads need to synchronize with one another to avoid race conditions



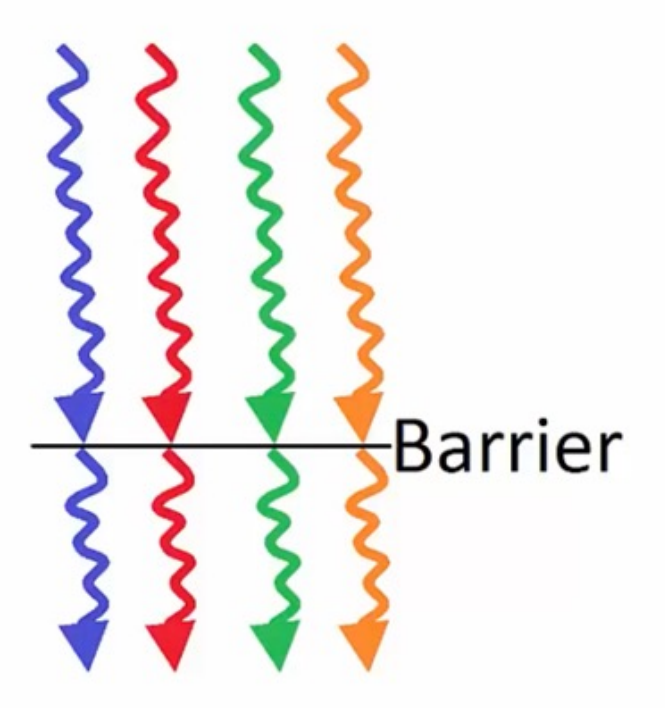
Thread Synchornization via Explicit Barrier

- Threads need to synchronize with one another to avoid race conditions
- A barrier is a point in the kernel where all the threads stop and wait on the others



Thread Synchornization via Explicit Barrier

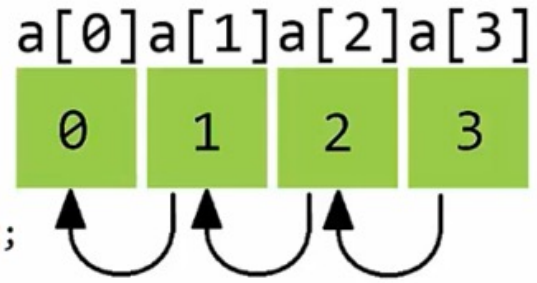
- Threads need to synchronize with one another to avoid race conditions
- A barrier is a point in the kernel where all the threads stop and wait on the others
- When all threads have reached the barrier, they can proceed
- Barriers can be implemented with:
  - __syncthreads();



syncthreads() Example

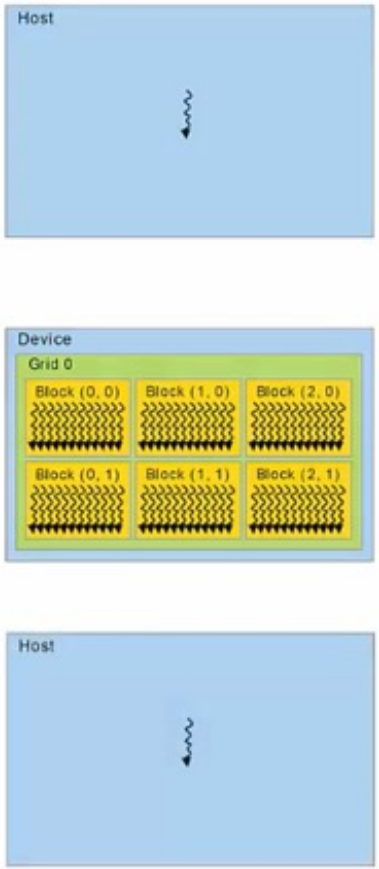
- Goal: Shift the contents of an array to the left by one elements

```
// Kernel Definition
__global__ void kernel( int* a )
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if( i < 3 )
    {
        int temp = a[i+1];
        __syncthreads;
        a[i] = temp;
        __syncthreads;
    }
}
```

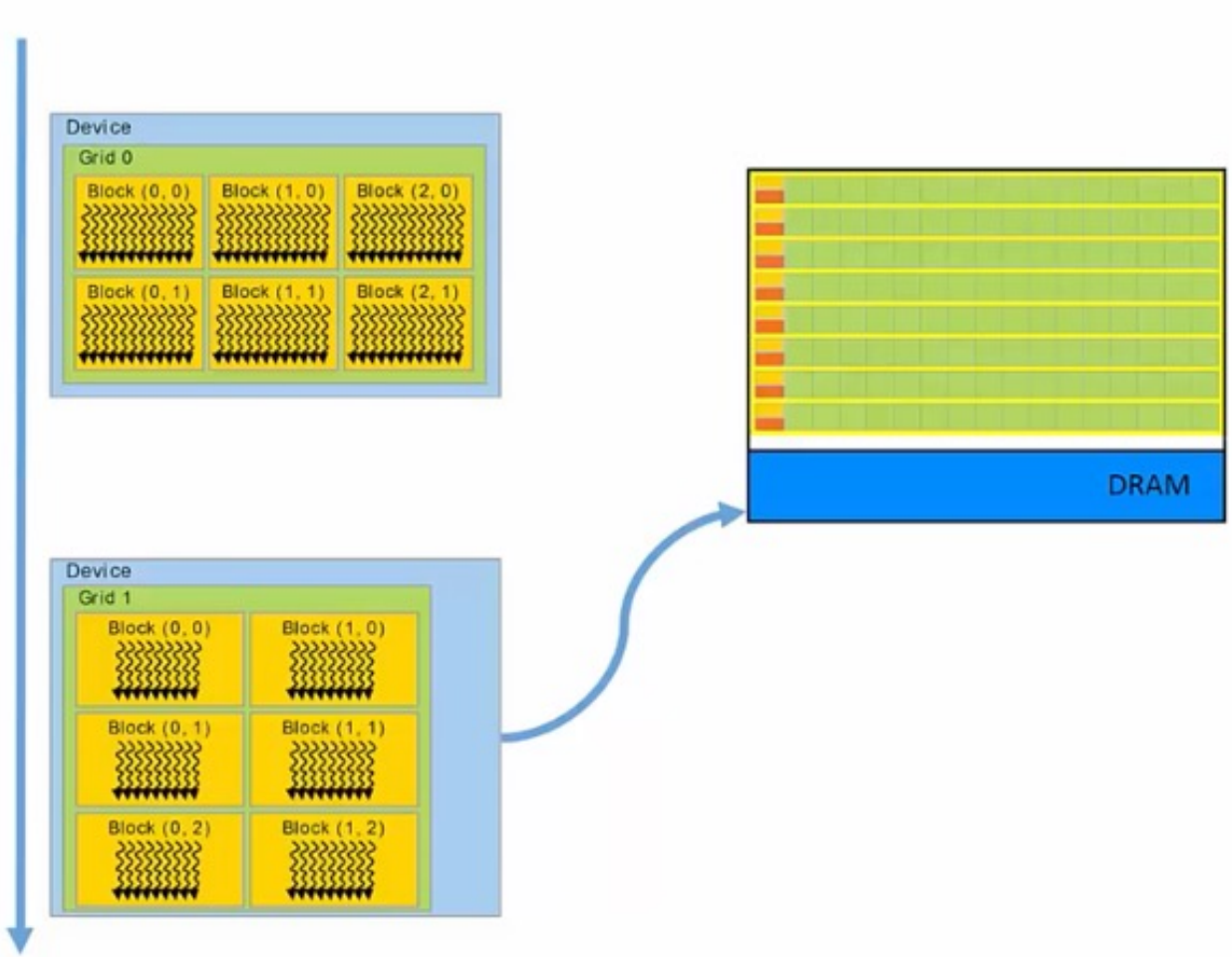


Implicit Barriers between Kernels

```
int main( void ) { // Host Code

  // Do sequential stuff

  // Launch Kernel
  kernel_0<<<grid_sz0,blk_sz0>>>(...);

  // Force Host to wait on the
  // completion of the Kernel
  cudaDeviceSynchronize();

  // Copy data from Device to Host
  cudaMemcpy(...);

  // Do sequential stuff
  // ...

  return 0;
}
```



Implicit Barriers between Kernels



Lesson Outline

- Heterogenous architectures motivations
- Hierarchy of Computations:
  - Threads
  - Blocks
  - Grids
- Corresponding Memory Spaces
  - Local
  - Shared
  - Global
- Synchronization Primitives
  - Implicit Barriers
  - Thread Synchronization
- NVIDIA GPUs and CUDA:
  - Compute capability

- CUDA Hardware
- CUDA Compilation and Runtime:
  - CUDA Runtime, CUDA Driver, AoT and JIT compilation
- CUDA Programming Model:
  - Grid, Block, Thread
  - UVM
- CUDA Warp Scheduling
- Context and Stream
- CUDA Profiling and Debugging

Acknowledgments

Part of this material has been adapted from the “The GPU Teaching Kit” that is licensed by NVIDIA and the University of Illinois under the Creative Commons Attribution-NonCommercial 4.0 International License.

Part of the material has been adopted from Josh Holloway (CUDA Teaching Center), Prof. Dr. Juan Gómez Luna, and Prof. Onur Mutlu from ETH Zürich